

FACULDADE
FACINT





E-book

ORGANIZAÇÃO E MANIPULAÇÃO EM ESTRUTURA DE DADOS

www.facint.edu.br



FACULDADE FACINT

MISSÃO

"Formar profissionais éticos, inovadores e preparados para transformar a sociedade, promovendo o desenvolvimento humano por meio de uma educação superior de excelência, com foco em tecnologia, gestão e responsabilidade social"

VISÃO

"Ser uma Instituição de Ensino Superior de referência educacional, reconhecida pela qualidade do ensino presencial e a distância, pela valorização do ser humano e pela sua atuação transformadora na sociedade VISÃO Ser uma Instituição de Ensino Superior de referência educacional, reconhecida pela qualidade do ensino presencial e a distância, pela valorização do ser humano e pela sua atuação transformadora na sociedade."

VALORES

I. Ética e integridade em todas as relações;
II. Valorização do ser humano como centro do processo educativo;
III. Comprometimento com a qualidade acadêmica e administrativa;
IV. Inovação e criatividade como pilares do aprendizado;
V. Inclusão e diversidade, respeitando as diferentes realidades e culturas;
VI. Desenvolvimento sustentável, com responsabilidade social e ambiental;
VII. Autonomia, liberdade de pensamento e expressão;
VIII. Educação transformadora, que forma cidadãos críticos, solidários e atuantes;
IX. Parceria com o setor produtivo para alinhamento entre teoria e prática;
X. Espírito colaborativo e diálogo aberto com toda a comunidade acadêmica;

FACULDADE FACINT

CAPÍTULO 1

INTRODUÇÃO A ALGORITMOS DE ORDENAÇÃO

Capítulo 1
Introdução a Algoritmos de Ordenação

Introdução

Neste primeiro capítulo, você será convidado a dar um passo fundamental no estudo das Estruturas de Dados: a compreensão dos algoritmos de ordenação. Antes de avançar para estruturas mais complexas e técnicas sofisticadas, é essencial entender como os dados podem ser organizados de forma eficiente, lógica e adequada a diferentes cenários computacionais.

A ordenação de dados está presente em inúmeras aplicações do nosso cotidiano — desde listas simples, como contatos em um celular, até sistemas robustos, como bancos de dados, motores de busca e plataformas digitais. Por isso, compreender como esses algoritmos funcionam, quais são suas propriedades e em que contextos devem ser utilizados é uma habilidade indispensável para quem atua ou pretende atuar na área de computação e engenharia de software.

Ao longo deste capítulo, você conhecerá os conceitos fundamentais de ordenação, suas principais características e critérios de análise, como desempenho e consumo de recursos. Também será apresentada a noção de complexidade assintótica, por meio da notação Big-O, permitindo comparar algoritmos e avaliar sua eficiência em diferentes situações. Além disso, serão estudados algoritmos clássicos de ordenação, com destaque para aqueles de menor complexidade cognitiva, que servem como base para o desenvolvimento do raciocínio lógico e algorítmico.

Este capítulo foi estruturado para que você não apenas aprenda *como* ordenar dados, mas principalmente *por que* determinadas soluções são mais adequadas do que outras. Ao final da leitura, espera-se que você seja capaz de analisar problemas reais, compreender o impacto da ordenação no desempenho de sistemas e aplicar os algoritmos estudados de forma consciente e fundamentada.

Prepare-se para iniciar essa jornada, que será essencial para os próximos capítulos e para toda a sua formação em Estruturas de Dados.

1. Ordenação de Dados - Conceito

Em muitas ocasiões, há uma necessidade eminente de se ordenar valores, objetos e até mesmo pessoas, a depender da ocasião. Na Ciência da Computação, também há a necessidade de ordenar. Muitas vezes, essa necessidade de ordenação é reflexo da necessidade de uma outra área, e que também tem um propósito definido.

Segundo Laureano (2008), ordenação pode ser definida como o processo de arranjar um conjunto de informações similares em uma ordem ascendente ou descendente. Podemos pensar em um arranjo de dados, referentes a registros de pessoas, veículos ou objetos, e que precisam ser ordenados com base em um ou mais critérios, como ordem de chegada, nome, descrição, documento, entre outros. Podemos dizer, então, que uma ordenação - em Ciência da Computação - é um algoritmo responsável por atribuir uma sequência em uma coleção - ou arranjo - de elementos, efetuando a sua ordenação parcial ou total.

Há infinitos cenários em que a ordenação do conjunto de dados disponíveis tem grande utilidade, e podemos citar um fragmento minúsculo desses cenários, como:

- Registros em relatórios (clientes por nome, veículos por UF, vendas por data);
- Filas de atendimento (ordem de chegada, prioridade, urgência do atendimento) ;
- Organização de informações (listas telefônicas, agendas de contatos);
- *E-commerces* e sistemas de vendas (produtos mais vendidos, produtos mais baratos/caros);
- Serviços financeiros (classificações de transações por data, avaliação de crédito);

- Redes sociais e plataformas de streaming (feed de notícias e sistemas de recomendações)
- Jogos e competição (placar de jogos, torneios, classificação);
- Sistemas operacionais (ordenação de tarefas);
- Processamento de linguagem neural (dicionários de palavras);;;;;;
- Pesquisas de dados (ordenação de tabelas e coleções em bancos de dados SQL/NoSQL);

Há inúmeros motivos para se ordenar uma sequência, principalmente para tornar sua busca mais eficiente durante a iteração de seus elementos. Cognitivamente os seres humanos interage mais rápido com coleções organizadas, optando sempre por rearranjá-las sempre com base em um critério específico ou mesmo seu valor absoluto. E isso não é diferente com computadores. Não somente a busca, mas qualquer operação se torna mais rápida com coleções de dados ordenadas, evitando buscas repetitivas por dados em sequências no algoritmo principal e, consequentemente, reduzindo o peso computacional do algoritmo principal.

No que se refere à base de dados em SGBDs relacionais, os algoritmos de ordenação desempenham um papel crucial nas operações com registros, atuando em operações de pré-classificação em que os resultados serão utilizados durante a busca de registros.

1.2. Ordenação de Dados - Propriedades

Podemos determinar que uma operação de ordenação classifica os elementos em um arranjo, atribuindo-lhes posições bem definidas e organizadas de acordo com um critério previamente estabelecido. Esse critério pode ser numérico, alfabético, cronológico ou baseado em qualquer atributo de um objeto, como idade, preço, data ou prioridade.

O resultado do processo de ordenação é um novo arranjo (ou o mesmo arranjo reorganizado) no qual os elementos passam a ocupar posições específicas e previsíveis, facilitando operações posteriores, como busca, inserção e análise dos dados.

Logo, em um arranjo ordenado ascendentemente (crescente), temos:

$$i_1 \leq i_2 \leq \cdots \leq i_n$$

Consequentemente, em um arranjo ordenado descendentemente (decrescente), temos:

$$i_1 \geq i_2 \geq \cdots \geq i_n$$

Exemplo:
Considere o arranjo de números inteiros:

[7, 3, 10, 1, 5] [7,\ 3,\ 10,\ 1,\ 5] [7, 3, 10, 1, 5]

Após a ordenação ascendente, o arranjo resultante será:

[1, 3, 5, 7, 10] [1,\ 3,\ 5,\ 7,\ 10] [1, 3, 5, 7, 10]

Nesse caso, cada elemento ocupa uma posição tal que nenhum valor à sua esquerda é maior que ele. Esse tipo de ordenação é amplamente utilizado, por exemplo, na organização de preços do mais barato para o mais caro ou na classificação de alunos por nota crescente. Em um arranjo ordenado descendentemente (ordem decrescente), os elementos são organizados do maior para o menor valor.

1.3. Algoritmos de Ordenação

Há também uma porção de algoritmos de ordenação disponíveis, que podem ser implementados independentemente da linguagem de programação,

ou mesmo serem executadas diretamente por um ser humano. A quantidade de algoritmos disponíveis para ordenação é meramente um reflexo das inúmeras maneiras de se rearranjar uma coleção de objetos e dados.

Muitos desses algoritmos preocupam-se prioritariamente com o tempo computacional, porém outros visam a otimização de recursos de memória, assim como tem aqueles cuja preocupação é a facilidade de implementação.

Podemos classificar os algoritmos de ordenação de algumas formas, avaliando alguns critérios como:

- Tempo de Execução (complexidade temporal)
- Espaço de memória (complexidade espacial)
- Estabilidade (manutenção da ordem relativa de elementos iguais)
- *In-place* (uso de memória adicional mínima).

Em qualquer um desses casos, sempre se avalia o comportamento do algoritmo em relação à sua complexidade assintótica em dois cenários possíveis: o melhor caso e o pior caso. Em ambos cenários, podemos usar a notação *Big-O* para descrever o comportamento do algoritmo, analisando sua escalabilidade em comparação com os demais algoritmos de ordenação conhecidos.

1.4. Algoritmos de Ordenação - *Big-O*

A notação *Big-O*, ou notação Bachmann-Landau, ou também conhecida como notação assintótica, é a notação matemática que descreve o comportamento limitante de uma função quando o argumento tende a um valor específico ou ao infinito.

Pertence a uma família de notações inventadas por Paul Bachmann, Edmund Landau e outros. De maneira simplificada, a notação *Big-O* descreve a complexidade do seu código usando termos algébricos, como o típico exemplo $O(n^2)$, também conhecida como *Big-O* quadrática.

Na expressão $O(n^2)$, temos o termo n representando o tamanho da entrada - sendo a quantidade de elementos da ordenação, enquanto a função $g(n)=n^2$, interna ao O(), nos dá a ideia de complexidade do algoritmo em relação ao tamanho da entrada.

No contexto da ordenação, notação *Big-O* serve principalmente para compararmos o tempo computacional entre diferentes algoritmos, compreendendo suas vantagens e desvantagens e sua praticidade no melhor caso e no pior caso.

Embora tenhamos critérios bem objetivos que podem vir a servir de classificação para os Algoritmos de Ordenação, nesta ocasião, adotaremos outro critério para classificação: a complexidade cognitiva.

Implementação Simplificada	Implementação Complexa
• **Insertion Sort** • **Bubble Sort** • **Selection Sort**	• **Quick Sort** • **Heap Sort** • **Merge Sort**

Na primeira parte desta disciplina, analisaremos a implementação e o comportamento dos algoritmos de implementação simplificada, isto é, com complexidade cognitiva baixa. Posteriormente, faremos a mesma coisa, porém para os algoritmos com maior dificuldade, ou complexidade cognitiva alta.

1. 5. 2. *Insertion Sort*

O *Insertion Sort* - em português “Ordenação por Inserção” - é um algoritmo simples de ser implementado, sendo considerado por muitos o mais simples entre os algoritmos de ordenação, possuindo a mais baixa complexidade cognitiva.

Sua lógica é inspirada no modo como os seres humanos organizam cartas em um jogo de baralho: a cada nova carta recebida, ela é inserida na posição correta em relação às cartas já ordenadas.

Este algoritmo costuma ser indicado para listas pequenas de valores a serem ordenados, uma vez que possui complexidade temporal elevada, e realiza muitas comparações.

Em contrapartida, possui implementação simplificada, uma vez que exige baixa complexidade cognitiva do programador, servindo muitas vezes - como agora - de mecanismo didático para ensinar algoritmos de ordenação.

Exemplo:
Considere o arranjo de inteiros:

[8, 3, 5, 2, 9] [8,\ 3,\ 5,\ 2,\ 9] [8, 3, 5, 2, 9]

- Inicialmente, o elemento **8** é considerado ordenado;
- O elemento **3** é comparado com **8** e inserido antes dele → [3, 8, 5, 2, 9][3,\ 8,\ 5,\ 2,\ 9][3, 8, 5, 2, 9];
- O elemento **5** é inserido entre **3** e **8** → [3, 5, 8, 2, 9][3,\ 5,\ 8,\ 2,\ 9][3, 5, 8, 2, 9];
- O elemento **2** é inserido na primeira posição → [2, 3, 5, 8, 9][2,\ 3,\ 5,\ 8,\ 9][2, 3, 5, 8, 9];
- O elemento **9** já está na posição correta.

Ao final do processo, o arranjo encontra-se completamente ordenado. Do ponto de vista computacional, o *Insertion Sort* apresenta **baixa complexidade cognitiva**, ou seja, é fácil de entender, implementar e depurar. No entanto, essa simplicidade vem acompanhada de um custo computacional elevado em grandes volumes de dados, pois o algoritmo realiza diversas comparações e movimentações de elementos.

Propriedades do Insertion Sort

O algoritmo Insertion Sort possui as seguintes características principais:

- **Estável:** mantém a ordem relativa de elementos com valores iguais.
- **In-place:** utiliza apenas uma quantidade constante de memória adicional.
- **Adaptativo:** apresenta melhor desempenho quando o arranjo já está parcialmente ordenado.
- **Iterativo:** não utiliza recursão, o que facilita sua implementação.

Essas propriedades fazem com que o *Insertion Sort* seja bastante eficiente para listas pequenas ou quase ordenadas, sendo inclusive utilizado como parte de algoritmos híbridos em bibliotecas padrão de linguagens de programação.

Análise Assintótica

A eficiência do *Insertion Sort* pode ser analisada considerando três cenários:

Por ser um algoritmo que executa muitas iterações, o número de comparações no **pior caso** ocorre quando o arranjo está ordenado de forma inversa. Nesse caso, cada novo elemento precisa ser comparado com todos os anteriores, resultando em uma complexidade temporal de é: $\mathbf{O(n^2)}$

O **melhor caso** ocorre quando o arranjo já está ordenado. O algoritmo realiza apenas uma comparação por elemento. O número de operações é proporcional a $2(n-1)$, mas assintoticamente a complexidade é **O(n)**.

No **caso médio**, assumimos que os elementos estão em **ordem aleatória** e em média, cada elemento precisa "andar" metade do vetor já ordenado. Ou seja: para o elemento na posição i, são feitas cerca de $i/2$

comparações/deslocamentos. Ou seja, o número de operações cresce **quadraticamente**.

No caso médio, cada elemento percorre, em média, metade da parte já ordenada do vetor, resultando em aproximadamente $n^2/4n$ operações, o que leva a uma complexidade assintótica **$O(n^2)$**.

Resumindo:

Caso	Complexidade
Melhor	O(n)
Médio	$O(n^2)$
Pior	$O(n^2)$

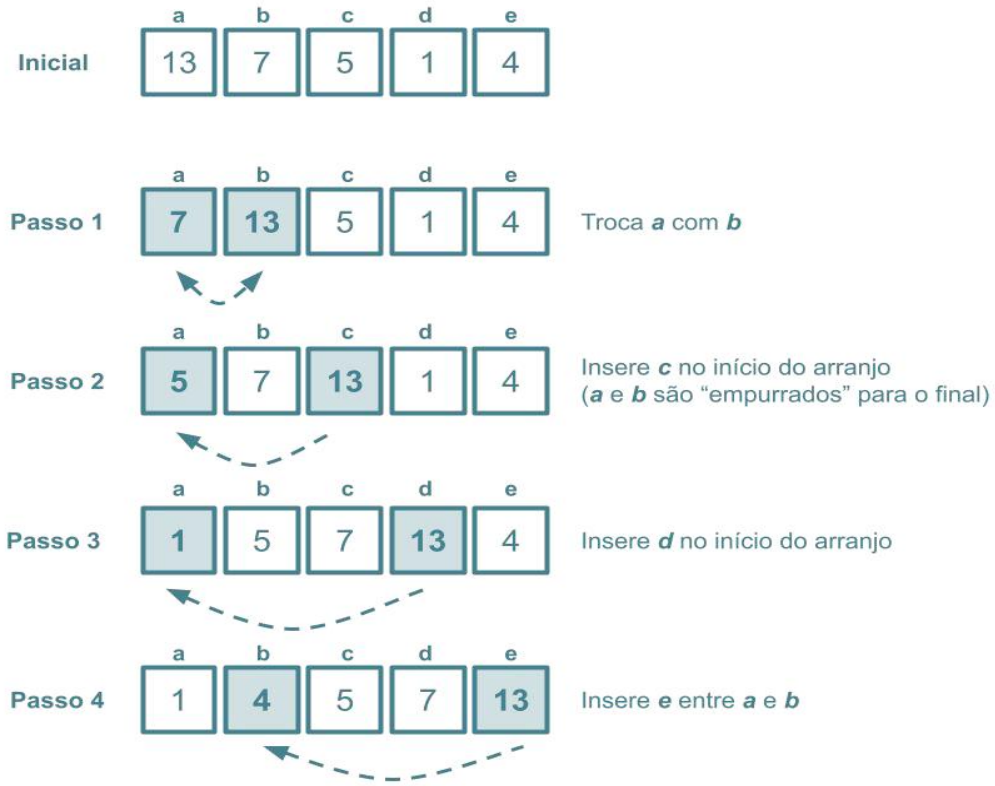
A ordenação por inserção pode ser representada por:

```
Input: A tabela (B) a ser ordenada
Output: A tabela (B) ordenada
begin
    for i = 2 to comprimento de B do
        valor ← B[i]
        insira B na sequência ordenada de B[1...i - 1]
        j ← i - 1
            while j > 0 E B[j] > valor do
                troque B[j + 1] com B[j]
                j ← j - 1
            endw
    endfor
    return B
end
```

O funcionamento do *Insertion Sort* baseia-se na construção gradual de uma sequência ordenada. O algoritmo considera que o primeiro elemento do arranjo já está ordenado e, a partir do segundo elemento, passa a inserir cada novo valor na posição correta dentro da parte já ordenada do vetor. Esse

processo se repete até que todos os elementos tenham sido analisados e posicionados adequadamente.

Figura 1 - Sequência de ordenação por inserção.



Fonte: Laureano (2008) - Adaptado.

Pseudocódigo

código

```
PROGRAMA "INSERTION_SORT"
VARIÁVEIS
   i, j, valor, temporário: INTEIRO
   arranjo: VETOR[5] DE INTEIRO
INÍCIO ALGORITMO
    PARA i DE 2 ATÉ TAMANHO(arranjo)-1 FAÇA
        valor ← arranjo[i]
            j ← i-1

            ENQUANTO (j > 0 E arranjo[j] > valor) FAÇA
                  temporário ← arranjo[j]
                  arranjo[j] ← valor
                  valor ← temporário
                  j ← j - 1
            FIM ENQUANTO
      FIM PARA
FIM ALGORITMO
```

CAPÍTULO 2

BUBBLE SORT, SELECTION SORT

Capítulo 2

Bubble Sort, *Selection Sort*

Introdução

Após a compreensão inicial dos algoritmos de ordenação e da análise do Insertion Sort, este capítulo dá continuidade ao estudo dos métodos de ordenação de **baixa complexidade cognitiva**, apresentando dois algoritmos clássicos e amplamente conhecidos: ***Bubble Sort*** e ***Selection Sort***.

Ambos os algoritmos compartilham características importantes para o aprendizado em Estruturas de Dados: são simples de entender, fáceis de implementar e permitem visualizar claramente o funcionamento das operações de comparação e troca. Embora não sejam eficientes para grandes volumes de dados, desempenham um papel fundamental no processo de formação do raciocínio algorítmico, servindo como base para a compreensão de técnicas mais avançadas de ordenação.

Neste capítulo, o estudante será conduzido à análise do funcionamento interno de cada algoritmo, observando como os elementos são comparados, reposicionados e organizados ao longo das iterações. Além disso, será apresentada a **análise assintótica** de ambos os métodos, considerando os cenários de melhor caso, caso médio e pior caso, reforçando a aplicação prática da notação Big-O.

Ao final deste capítulo, espera-se que o leitor seja capaz de:

- Compreender o comportamento do Bubble Sort e do Selection Sort;
- Identificar suas vantagens e limitações;
- Comparar seu desempenho com outros algoritmos já estudados;

- Reconhecer em quais cenários esses algoritmos podem ser utilizados de forma didática ou prática.

Esse aprofundamento prepara o terreno para os próximos capítulos, nos quais serão explorados algoritmos de ordenação mais eficientes e estratégias computacionais mais sofisticadas, ampliando a capacidade de análise e escolha adequada de algoritmos para diferentes problemas.

2.1. *Bubble Sort*

O algoritmo de ordenação ***Bubble Sort*** (traduzido livremente como *ordenação por bolha*) é um método simples de ordenação baseado em **trocas sucessivas entre elementos adjacentes**. Trata-se de um dos algoritmos de ordenação mais conhecidos e difundidos, tanto pelo seu nome intuitivo e de fácil memorização quanto pela sua lógica simples, que permite rápida compreensão mesmo por estudantes iniciantes em programação e Estruturas de Dados.

Apesar de sua popularidade, *o Bubble Sort* não é considerado um algoritmo eficiente, sobretudo quando aplicado a conjuntos de dados de grande porte. Em termos de desempenho computacional, ele figura entre os algoritmos menos eficientes, uma vez que realiza um elevado número de comparações e trocas, apresentando comportamento quadrático na maioria dos cenários. Por esse motivo, é frequentemente citado na literatura como um exemplo clássico de algoritmo de ordenação com baixa eficiência temporal.

Em contrapartida, o *Bubble Sort* se destaca por possuir baixa complexidade cognitiva, ou seja, é extremamente simples de ser entendido, aplicado e implementado. Essa característica o torna particularmente útil em contextos educacionais, sendo amplamente utilizado como ferramenta

pedagógica para introduzir conceitos fundamentais de ordenação, laços de repetição, comparações e trocas de elementos em um arranjo.

O funcionamento do *Bubble Sort* é baseado em dois laços de repetição aninhados. O laço mais externo é responsável por percorrer o arranjo repetidas vezes — geralmente n − 1 iterações, onde *n* representa o número de elementos do conjunto. Já o laço interno realiza as comparações entre elementos adjacentes, efetuando trocas sempre que identifica que os valores estão fora da ordem desejada. A cada iteração completa, o maior (ou menor, dependendo do critério) elemento "flutua" gradualmente para a extremidade do arranjo, comportamento que dá origem ao nome do algoritmo.

Dessa forma, embora o *Bubble Sort* não seja indicado para aplicações que demandam alto desempenho, ele desempenha um papel importante no ensino de algoritmos, servindo como base conceitual para a compreensão de métodos de ordenação mais sofisticados e eficientes.

Análise Assintótica

O algoritmo ***Bubble Sort*** apresenta comportamento semelhante nos cenários de **caso médio** e **pior caso**, uma vez que sua estratégia de ordenação baseia-se em comparações sucessivas entre elementos adjacentes, independentemente da disposição inicial do arranjo.

O **pior caso** ocorre quando o vetor está ordenado de forma **decrescente**, ou seja, completamente inverso à ordem desejada. Nessa situação, todas as comparações possíveis são realizadas, bem como o maior número de trocas. Assim, a complexidade temporal no pior caso é: $\mathbf{O(n^2)}$

O **melhor caso** ocorre quando o vetor já está **ordenado**. Caso o algoritmo seja implementado com uma verificação de trocas (flag de controle), ele pode encerrar a execução antecipadamente ao perceber que nenhuma troca foi

realizada durante uma passagem completa pelo vetor. Nesse cenário, o número de comparações é linear, resultando em complexidade temporal: **O(n).**

No **caso médio**, considera-se que os elementos do arranjo estão dispostos de forma **aleatória**. Ainda assim, o algoritmo executa praticamente o mesmo número de comparações do pior caso, realizando trocas sempre que encontra elementos fora de ordem. Dessa forma, a complexidade temporal no caso médio também é: **$O(n^2)$.**

Resumindo:

Caso	Complexidade
Melhor	O(n)*
Médio	$O(n^2)$
Pior	$O(n^2)$

* Considerando a versão otimizada com verificação de troca.

A ordenação por *Bubble Sort* pode ser representada por:

```
Input: A tabela (B) ser ordenada
Output: A tabela (B) ordenada
begin
        for i=comprimento de B to 1 do
                for j=1 to i -1 do
                        if B[j] > B[j+1] then
                                troque B[j] com B[j+1]
                        endif
                endfor
        endfor
end
```



Pseudocódigo

código

```
1  PROGRAMA "BUBBLE_SORT"
2  VARIÁVEIS
3        i, j, temporário : INTEIRO
4        arranjo : VETOR[5] DE INTEIRO
5  INÍCIO ALGORITMO
6
7        arranjo ← [10, 9, 7, 13, 5]
8
9        PARA i DE TAMANHO(arranjo) ATÉ 1 PASSO -1 FAÇA
10             PARA j DE 1 ATÉ i-1 FAÇA
11                   SE arranjo[j] > arranjo[j+1] ENTÃO
12                         temporário ← arranjo[j]
13                         arranjo[j] ← arranjo[j+1]
14                         arranjo[j+1] ← temp
15                   FIM SE
16             FIM PARA
17       FIM PARA
18 FIM ALGORITMO
```

2.2. *Selection Sort*

O ***Selection Sort*** (ou, em uma tradução livre, *Ordenação por Seleção*) é um algoritmo de ordenação baseado em **comparações**, classificado como ***in-place***, isto é, realiza a ordenação utilizando apenas uma quantidade **constante de espaço adicional**, além daquele já destinado ao armazenamento dos dados de entrada. Assim como os algoritmos de ordenação apresentados anteriormente, ele não requer estruturas auxiliares complexas, o que contribui para sua simplicidade de implementação.

O princípio fundamental do *Selection Sort* consiste em selecionar repetidamente o elemento que atende a um critério específico — geralmente o menor ou o maior valor do conjunto — e posicioná-lo em sua posição definitiva no arranjo. Inicialmente, o algoritmo percorre todo o vetor em busca do menor (ou maior) elemento e, ao encontrá-lo, realiza a troca com o elemento que ocupa a primeira (ou última) posição do arranjo. Dessa forma, estabelece-se corretamente a posição de um elemento a cada iteração.

Após a definição da primeira posição, o algoritmo restringe seu escopo ao subconjunto restante de elementos, excluindo aqueles que já foram posicionados corretamente. Em seguida, o processo é repetido: o vetor é novamente percorrido para identificar o próximo menor (ou maior) elemento, que então é reposicionado. Esse procedimento se repete sucessivamente até que todos os elementos estejam ordenados.

A ordenação por seleção apresenta complexidade cognitiva baixa, sendo considerada de fácil compreensão e implementação, especialmente em ambientes educacionais. Seu funcionamento é bastante intuitivo, permitindo que o estudante acompanhe claramente o processo de escolha e reposicionamento dos elementos a cada etapa da execução.

Entretanto, apesar de sua simplicidade, o *Selection Sort* executa um número elevado de comparações, independentemente da disposição inicial dos dados. Mesmo nos cenários em que o arranjo já se encontra parcialmente ou totalmente ordenado, o algoritmo percorre todos os elementos restantes a cada

iteração. Por esse motivo, ele é classificado como um algoritmo de complexidade temporal alta, apresentando desempenho quadrático, o que o torna pouco eficiente para ordenação de grandes volumes de dados.

Ainda assim, o *Selection Sort* possui grande relevância didática, servindo como um importante instrumento para o entendimento dos conceitos básicos de ordenação, análise de algoritmos e comportamento assintótico, além de preparar o estudante para o estudo de métodos de ordenação mais eficientes e sofisticados.

Análise Assintótica

O algoritmo *Selection Sort* apresenta um comportamento bastante previsível em relação ao número de comparações realizadas, independentemente da disposição inicial dos elementos no arranjo. Isso ocorre porque, a cada iteração, o algoritmo percorre todo o subconjunto de elementos ainda não ordenados para encontrar o menor (ou maior) valor, mesmo que o vetor já esteja parcialmente ou totalmente ordenado.

O **pior caso** ocorre quando o vetor está ordenado de forma **inversa** ao critério desejado. No entanto, diferentemente de outros algoritmos de ordenação, o *Selection Sort* executa praticamente o mesmo número de comparações em qualquer cenário. O número total de comparações realizadas é dado por:

$$(n-1)+(n-2)+\cdots 2+1=0\,(\frac{n(n-1)}{2})$$

Assim, a complexidade temporal no pior caso é: $O(n^2)$.

No **caso médio**, assume-se que os elementos do arranjo estão distribuídos de forma **aleatória**. Ainda assim, o algoritmo continua realizando a varredura completa dos elementos não ordenados em cada iteração, resultando no mesmo crescimento quadrático do número de comparações. Portanto, a complexidade temporal no caso médio é: $O(n^2)$.

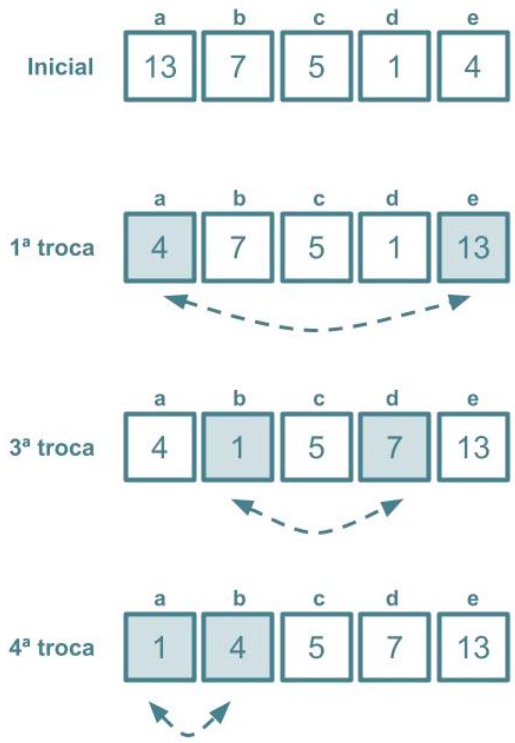
No **Melhor Caso,** mesmo quando o vetor já se encontra **totalmente ordenado**, o *Selection Sort* não se beneficia dessa condição, pois continua buscando o menor (ou maior) elemento em todo o subconjunto restante a cada iteração. Dessa forma, a complexidade temporal no melhor caso também é: $O(n^2)$.

Resumindo:

Caso	**Complexidade**
Melhor	O(n²)
Médio	O(n²)
Pior	O(n²)

A ordenação por *Bubble Sort* pode ser representada por:

Input: A tabela (S) a ser ordenada
Output: Tabela (S) ordenada
begin
    for i=comprimento de S to 2 do
        Posição ← 1
        for j=1 to i do
            if S[j] > S[Posição] then
                Posição ← j
            endif
        endfor
        troque B[i] com B[Posição]
    endfor
end



Pseudocódigo

código

```
PROGRAMA "SELECTION_SORT"
VARIÁVEIS
	i, j, temporário, posição_max : INTEIRO
	arranjo : VETOR[5] DE INTEIRO
INÍCIO ALGORITMO
	arranjo ← [10, 9, 7, 13, 5]

	PARA i DE TAMANHO(arranjo) ATÉ 2 PASSO -1 FAÇA
		posição_max ← 1

		PARA j DE 1 ATÉ i FAÇA
			SE arranjo[j] > arranjo[j+1] ENTÃO
				posição_max ← j
			FIM SE
		FIM PARA

		temporário ← arranjo[j]
		arranjo[j] ← arranjo[posição_max]
		arranjo[posição_max] ← temporário
	FIM PARA
FIM ALGORITMO
```

CAPÍTULO 3

CENÁRIOS PARA APLICAÇÃO DOS ALGORITMOS

Capítulo 3

Cenários para Aplicação dos Algoritmos de Ordenação Complexos

Introdução

Nos capítulos anteriores, foram apresentados algoritmos de ordenação de **baixa complexidade cognitiva**, como *Insertion Sort*, *Bubble Sort* e *Selection Sort*. Esses métodos, embora simples de compreender e implementar, apresentam limitações significativas em termos de desempenho computacional, especialmente quando aplicados a conjuntos de dados com grande volume de elementos.

Neste capítulo, o foco desloca-se da implementação básica para a **análise dos cenários de aplicação** dos algoritmos de ordenação, destacando as situações em que os métodos de complexidade quadrática se tornam inadequados. A partir dessa reflexão, são introduzidos algoritmos de ordenação mais eficientes, capazes de lidar com grandes volumes de dados por meio de estratégias computacionais mais elaboradas.

O capítulo explora os motivos pelos quais algoritmos com complexidade $O(n^2)$ tornam-se ineficientes à medida que o tamanho da entrada cresce, reforçando a importância da **análise assintótica** como ferramenta essencial para a escolha adequada de algoritmos. Em seguida, são apresentados algoritmos de ordenação considerados mais complexos, tanto do ponto de vista da implementação quanto do raciocínio necessário para sua compreensão, mas que oferecem ganhos significativos de desempenho.

Além disso, será abordada a estratégia de **dividir para conquistar**, amplamente utilizada por algoritmos de ordenação eficientes, como *Quick Sort* e *Merge Sort*. Essa estratégia permite decompor um problema grande em partes menores e mais simples, facilitando sua resolução e contribuindo para a redução do custo computacional.

Ao final deste capítulo, espera-se que o estudante seja capaz de compreender a relação entre **complexidade algorítmica, volume de dados e desempenho**, reconhecendo a importância de selecionar o algoritmo mais adequado para cada contexto. Essa abordagem prepara o leitor para o estudo aprofundado dos algoritmos de ordenação complexos apresentados nas seções seguintes, ampliando sua capacidade de análise crítica e tomada de decisão no desenvolvimento de software.

3.1. Ineficiência dos algoritmos $O(n^2)$

Como já discutido em capítulos anteriores, os algoritmos de ordenação mais simples, tais como ***Bubble Sort***, ***Selection Sort*** e ***Insertion Sort***, apresentam limitações significativas quando aplicados à ordenação de **grandes volumes de dados**. Embora sejam amplamente utilizados em contextos educacionais e em situações muito específicas, esses algoritmos tornam-se rapidamente ineficientes à medida que o tamanho do conjunto de elementos aumenta.

Esses métodos se caracterizam por possuir **baixa complexidade cognitiva**, o que significa que são fáceis de compreender, implementar e depurar. No entanto, essa simplicidade vem acompanhada de um custo computacional elevado. No pior caso — e, em muitos cenários, também no caso médio — o número de comparações realizadas cresce proporcionalmente ao quadrado do tamanho da entrada, isto é, tende a $\mathbf{O(n^2)}$. Como consequência, o tempo de execução aumenta de forma significativa mesmo com acréscimos relativamente pequenos no número de elementos a serem ordenados.

Outro fator que contribui para a ineficiência desses algoritmos é o seu modelo de execução estritamente sequencial. As comparações e trocas são realizadas de forma linear, elemento a elemento, o que impede a exploração de técnicas mais avançadas de otimização, como recursividade, divisão do problema em subproblemas independentes e paralelismo. Dessa forma, esses

algoritmos não conseguem aproveitar adequadamente arquiteturas modernas de processamento, que contam com múltiplos núcleos e execução concorrente.

Além disso, em aplicações reais que envolvem grandes volumes de dados — como sistemas de banco de dados, processamento de grandes arquivos, aplicações financeiras ou análise de dados — o uso de algoritmos $O(n^2)$ pode resultar em tempos de resposta elevados, consumo excessivo de recursos computacionais e degradação do desempenho geral do sistema.

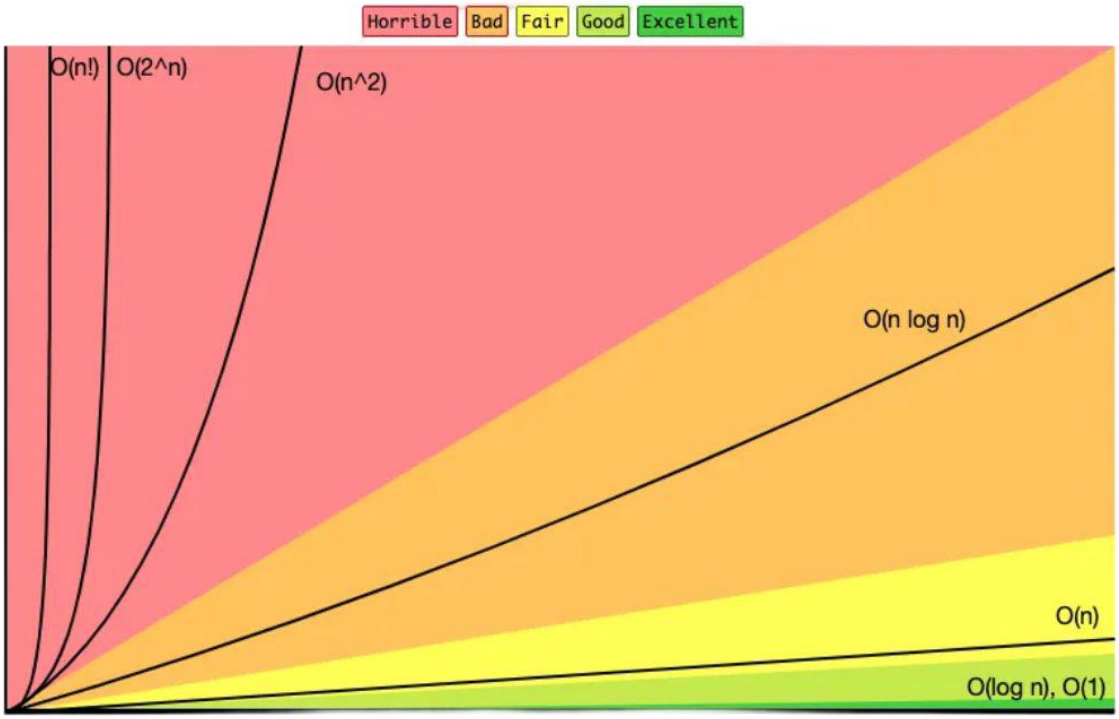
Portanto, embora os algoritmos de ordenação de complexidade quadrática desempenhem um papel fundamental no ensino dos conceitos básicos de algoritmos e estruturas de dados, sua utilização prática deve ser cuidadosamente avaliada. Em cenários que demandam eficiência e escalabilidade, torna-se indispensável a adoção de algoritmos de ordenação mais sofisticados, com melhor comportamento assintótico, tema que será explorado nas seções seguintes deste capítulo.

3.2. Algoritmos de Ordenação Complexos

Em grandes coleções de elementos, recomenda-se algoritmos de ordenação mais complexos, que utilizam outras estratégias para ordenação do conjunto. Na maioria dos casos, essas estratégias não são tão intuitivas, e são comumente pensadas para que o computador possa ordenar os elementos de acordo com sua capacidade.

Todavia a complexidade cognitiva do algoritmo aumenta significativamente, necessitando de uma atenção acurada do programador no momento de sua implementação, o que torna a compreensão do algoritmo muito mais dificultosa.

A grande vantagem dos algoritmos de ordenação complexos, em relação aos de complexidade cognitiva baixa, é a sua velocidade. As estratégias utilizadas por tais algoritmos otimizam o trabalho do computador, avaliando outros aspectos/propriedades dos elementos para realizar sua ordenação.



3.3. Estratégia Dividir para Conquistar

Uma parte considerável dos algoritmos de ordenação mais complexos utiliza uma estratégia interessante para solucionar o problema da ordenação: a divisão pela conquista. Diante de um grande problema, tanto humanos como computadores levam muito tempo para encontrar uma solução, mas, ao quebrar o problema em partes menores, mais simples de se entender, a solução pode ser encontrada de forma dividida, restando apenas juntar a solução do problema.

Nessa estratégia, pressupomos que o computador não sabe resolver o problema de por inteiro, sendo necessário a divisão até a menor parte possível, até que encontre a menor parte solucionável pelo algoritmo, sendo considerado, então, uma estratégia recursiva.

Principais Algoritmos de Ordenação

São inúmeros algoritmos de ordenação disponíveis para se trabalhar com grandes coleções de dados. Os mais citados e também utilizados costumam ser Quick Sort, Merge Sort, Heap Sort e Radix Sort, sempre variando em relação à sua aplicação e também complexidade assintótica.

Algoritmos de Ordenação Complexos			
Algoritmo	Pior Caso	Melhor Caso	Caso Médio
Quick Sort	$O(n^2)$	$O(n \log n)$	$O(n \log n)$
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$
Radix Sort	$O\ (n \times k)$	$O\ (n \times k)$	$O\ (n \times k)$

Os algoritmos de ordenação complexos têm uma ampla gama de aplicações em diversos campos da ciência da computação e da engenharia de software. Aqui estão algumas das principais aplicações para cada um dos algoritmos de ordenação complexos mencionados:

- Sistemas de Bancos de Dados relacionais;
- Sistemas Operacionais;
- Processamento de Textos;
- Ordenação de Listas Grandes;
- Sistemas de Arquivos e mídias digitais;
- Sistema de Tempo Real;
- Prioridade de Fila;
- Ordenação de *Strings*;
- Dados Científicos;
- Aplicações Financeiras;
- Sistemas Eleitorais.



A eficiência é uma característica que varia de algoritmo para algoritmo. Não somente é considerada a velocidade de ordenação, mas também o consumo de recursos de cada algoritmo, além da análise assintótica de cada cenário aplicado.

Muitos algoritmos têm destaque em relação à sua velocidade, como *Merge Sort* e o *Radix Sort*, porém seu consumo de memória pode ser considerado elevado, e até em alguns casos exacerbados para determinado cenário.

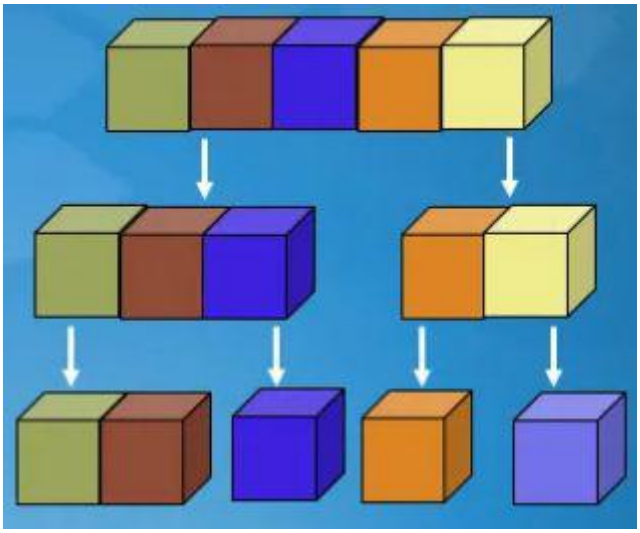
Por isso, a escolha do algoritmo de forma adequada, dimensionado ao cenário em questão, pode uma grande eficiência, levando-se em consideração capacidade de processamento e alocação de memória.

3.4. *Quicksort*

O *Quick Sort* é um algoritmo de ordenação do tipo divisão e conquista, desenvolvido por por C.A.R. Hoare em 1959 enquanto ele era um estudante visitante na Universidade Estadual de Moscou. Foi publicado posteriormente em 1962.

O algoritmo do *Quick Sort* é do tipo recursivo, e resolve a ordenação (problema) dividindo o arranho em partes menores (caso recursivo) até que haja apenas um único elemento em cada subarranjo (caso trivial).

O *Quick Sort* baseia-se na escolha de um número, que servirá de pivô para separação dos conjuntos à esquerda e à direita, sendo à esquerda menor ou igual ao pivô, e a direita maior ou igual ao pivô. Hoare escolheu o nome "*Quick Sort*" para refletir a eficiência e rapidez do algoritmo em comparação com outros métodos de ordenação conhecidos na época.



Complexidade do *Quick Sort*

O ***Quick Sort*** é um algoritmo de ordenação baseado na estratégia de **dividir para conquistar**, sendo amplamente reconhecido por sua eficiência prática e bom desempenho médio. Sua complexidade está diretamente relacionada à forma como o vetor é particionado a cada etapa do algoritmo, especialmente à escolha do **pivô**, elemento responsável por dividir o arranjo em subconjuntos menores.

Diferentemente dos algoritmos de ordenação de complexidade quadrática estudados anteriormente, o *Quick Sort* busca reduzir o tamanho do problema a cada chamada recursiva, reorganizando os elementos de modo que valores menores que o pivô fique à esquerda e valores maiores fiquem à direita. Esse processo de particionamento determina o comportamento assintótico do algoritmo.

O **melhor caso** ocorre quando o pivô escolhido divide o arranjo em duas partes aproximadamente iguais a cada iteração. Nesse cenário, o algoritmo realiza sucessivas divisões equilibradas, reduzindo significativamente o número de comparações necessárias.

A complexidade temporal no melhor caso é dada por:

$$O\ (n \log n)$$

Esse comportamento é resultado do fato de que o vetor é dividido em $\log_2(n)$ níveis de recursão, e em cada nível são realizadas O(n) comparações durante o particionamento.

No **caso médio**, assume-se que os elementos do arranjo estão distribuídos de forma aleatória e que o pivô, em média, produz divisões razoavelmente balanceadas. Mesmo sem garantir divisões perfeitamente iguais, o *Quick Sort* mantém um desempenho eficiente na maioria das situações práticas.

Assim, a complexidade temporal no caso médio também é:

$$O\ (n \log n)$$

Esse é o principal motivo pelo qual o *Quick Sort* é amplamente utilizado em bibliotecas padrão e aplicações reais, apesar de seu pior caso desfavorável.

O **pior caso** ocorre quando o pivô escolhido é sempre o menor ou o maior elemento do arranjo, resultando em divisões extremamente desbalanceadas. Nesse cenário, o algoritmo reduz o problema em apenas uma unidade a cada chamada recursiva, comportando-se de forma semelhante a algoritmos de ordenação quadráticos.

Nessa situação, a complexidade temporal no pior caso é: $O(n^2)$.

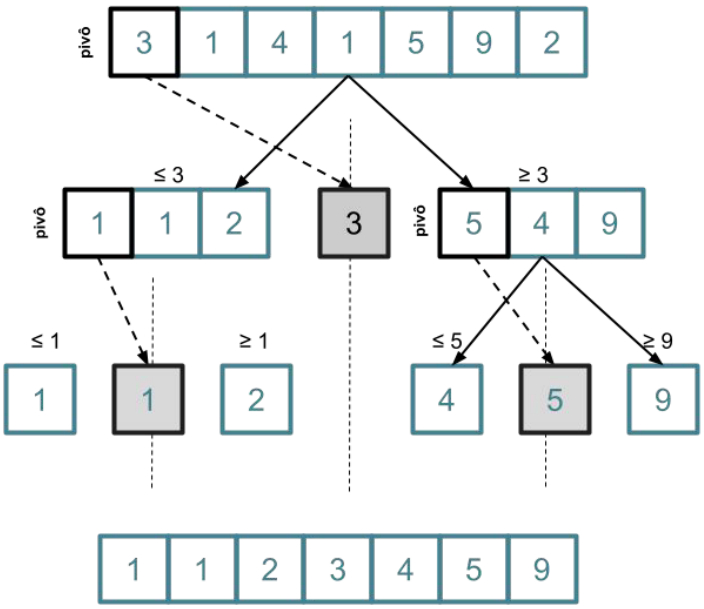
Esse comportamento pode ocorrer, por exemplo, quando o vetor já está ordenado e o pivô é sempre escolhido como o primeiro ou o último elemento.

Resumindo:

Caso	Complexidade Temporal
Melhor	O(n log n)
Médio	O(n log n)
Pior	$O(n^2)$

A ordenação por *Bubble Sort* pode ser representada por:

```
Input: Vetor V, índice inicial p, índice final r
Output: Vetor V ordenado
begin
    if p < r then
        q ← Particiona(V, p, r)
        QuickSort(V, p, q-1)
        QuickSort(V, q+1, r)
    endif
    return V
end
```



O *Quick sort* é aplicado, na maioria das vezes, para os seguintes cenários:

- Sistemas de Banco de Dados: Usado para operações de ordenação interna devido à sua eficiência média.
- Processamento de Textos: Para ordenação de palavras e linhas.
- Sistemas Operacionais: Para gerenciar listas de processos e recursos.

3.5. *Merge Sort*

Assim como o algoritmo ***Quick Sort***, o ***Merge Sort*** também utiliza a estratégia de **dividir para conquistar**, sendo classificado como um algoritmo de ordenação por **intercalação** ou **segmentação**. Essa abordagem parte do princípio de que um grande conjunto de dados representa um problema complexo demais para ser resolvido de forma direta e eficiente. Dessa maneira, o algoritmo decompõe o problema original em partes menores, mais simples e mais fáceis de ordenar.

No funcionamento do *Merge Sort*, pressupõe-se que o algoritmo não seja capaz de ordenar o arranjo completo de uma única vez. Assim, o vetor original é dividido fisicamente em subarranjos menores, que continuam sendo subdivididos recursivamente até que cada subarranjo contenha apenas um ou dois elementos. Nessa etapa, considera-se que o problema atingiu seu caso base ou trivial, pois um conjunto com um único elemento já está, por definição, ordenado, e conjuntos muito pequenos podem ser ordenados diretamente com facilidade.

Diferentemente do *Quick Sort*, o *Merge Sort* **não utiliza um pivô** para realizar a divisão dos dados. A segmentação do arranjo ocorre de forma sistemática, separando o vetor original em duas partes de tamanhos iguais — ou quase iguais, no caso de um número ímpar de elementos — independentemente dos valores contidos nos dados. Após a divisão, cada metade é ordenada recursivamente e, em seguida, ocorre a etapa fundamental do algoritmo: a **mesclagem (*merge*)**.

A fase de mesclagem consiste em intercalar os elementos dos subarranjos já ordenados, produzindo um novo arranjo também ordenado. Esse processo é realizado de maneira sequencial e controlada, comparando os elementos das duas partes e inserindo-os na ordem correta em um vetor auxiliar. Ao final das sucessivas mesclagens, obtém-se o arranjo totalmente ordenado.

O *Merge Sort* foi criado em 1945 pelo matemático e cientista da computação John von Neumann, o mesmo responsável pela concepção da Arquitetura de von Neumann, base da maioria dos computadores modernos. Desde sua criação, o algoritmo se destacou por apresentar desempenho consistente e previsível, mantendo a mesma complexidade temporal independentemente da disposição inicial dos dados, característica que o torna especialmente adequado para aplicações que lidam com grandes volumes de informação.

Complexidade do *Merge Sort*

O ***Merge Sort*** apresenta um comportamento assintótico **estável e previsível**, independentemente da disposição inicial dos elementos no arranjo. Isso ocorre porque o algoritmo sempre divide o vetor em partes aproximadamente iguais e executa a etapa de mesclagem de forma sistemática, sem depender de escolhas condicionais, como a seleção de pivôs.

Complexidade Temporal

O funcionamento do *Merge Sort* pode ser dividido em duas etapas principais:

1. **Divisão do arranjo** em subarranjos menores, realizada recursivamente;
2. **Mesclagem (*merge*)** dos subarranjos ordenados, efetuando comparações sequenciais.

A cada nível da recursão, o vetor é dividido ao meio, gerando aproximadamente **$\log_2(n)$** níveis. Em cada nível, o processo de mesclagem percorre todos os **n elementos** do arranjo.

Dessa forma, a complexidade temporal do Merge Sort é:

- **Melhor caso:** $O\ (n \log n)$;
- **Caso médio:** $O\ (n \log n)$;
- **Pior caso:** $O\ (n \log n)$.

Esse comportamento uniforme é uma das principais vantagens do *Merge Sort* em relação a outros algoritmos de ordenação, como o *Quick Sort*, que pode apresentar degradação significativa no pior caso.

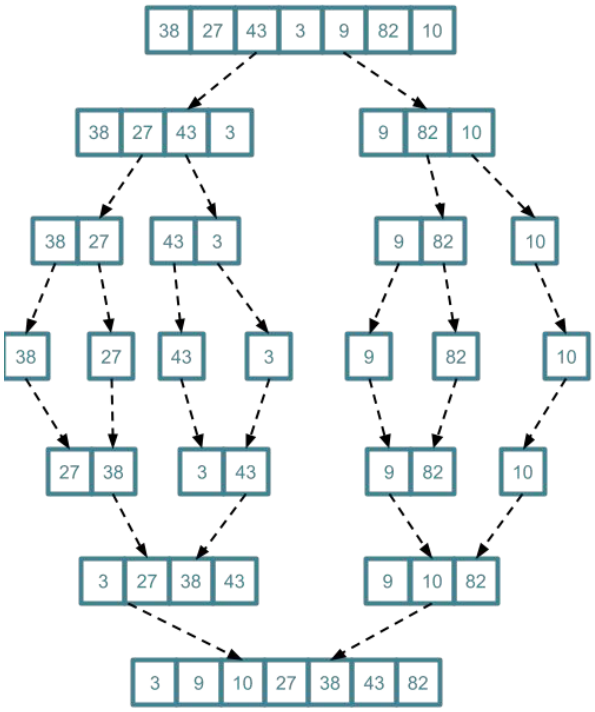
Resumindo:

Caso	Complexidade Temporal
Melhor	O(n log n)
Médio	O(n log n)
Pior	O(n log n)

A ordenação por *Merge Sort* pode ser representada por:

```
Input: Vetor V a ser ordenado
Output: Novo vetor R ordenado
begin
    if (TAMANHO(V) > 1) then
        meio ← TAMANHO(V)/2 + TAMANHO(V) MOD 2
        A ← V[1:meio]
        B ← V[meio+1:TAMANHO(V)]
        A ← MergeSort(A, TAMANHO(A))
        B ← MergeSort(B, TAMANHO(B))
        R ← Merge(A, B)
        return R
    endif
    return V
end
```

Figura 2 - Sequência de ordenação por *Merge Sort*.

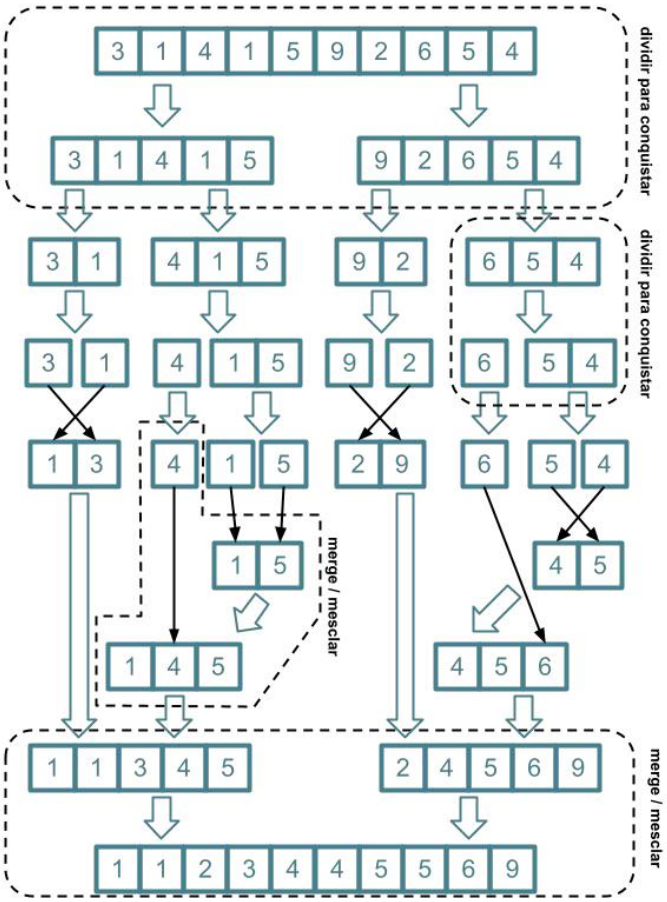


Fonte: Laureano (2008) - adaptado.

O processo de mesclagem pode ser entendido como:

```
Input: Vetores A e B
Output: Novo vetor M mesclado
begin
        tamanho_M ← TAMANHO(A) + TAMANHO(B)
        index_A, index_B ← 1, 1
        for i = 0 to tamanho_M do
                if (index_A > tamanhoA) then
                        Atribui B[index_B] em M e incrementa index_B
                else if (index_B > tamanho_B) then
                        Atribui A[index_A] em M e incrementa index_A
                else if
                        if (A[index_A] < B[index_B]) then
                                Atribui A[index_A] em M e incrementa index_A
                else
                                Atribui B[index_B] em M e incrementa index_B
                endif
                end if
        endfor
        return M
end
```

Figura 3 - Processo de Mesclagem do Merge Sort.



Fonte: Laureano (2008) - adaptado.

O *Merge Sort* é aplicado, na maioria das vezes, para os seguintes cenários:

- **Ordenação de Listas Grandes:** Ideal para ordenar grandes volumes de dados devido à sua eficiência e estabilidade.
- **Sistemas de Arquivos:** Para ordenação de arquivos muito grandes que não cabem na memória principal (ordenando em discos).
- **Aplicações de Mídia Digital**: Ordenação de grandes coleções de dados de mídia, como imagens e vídeos.

3.6. Radix Sort

O ***Radix Sort*** que pode ser traduzido como *ordenação por base numérica (x)*, é um algoritmo de ordenação **rápido e estável**, amplamente utilizado para ordenar arranjos de **elementos numéricos inteiros**. Diferentemente dos algoritmos de ordenação baseados em comparação, como *Quick Sort*, *Merge Sort* ou *Heap Sort*, o *Radix Sort* fundamenta-se na análise dos dígitos individuais dos elementos, o que lhe permite alcançar um desempenho bastante eficiente em determinados cenários. Além de números, o algoritmo também pode ser aplicado à ordenação de elementos identificados por **chaves únicas**, como códigos, identificadores ou cadeias de caracteres de comprimento fixo, desde que exista uma correspondência lexicográfica entre os símbolos analisados.

A origem da classificação por radix remonta ao ano de 1887, a partir dos trabalhos desenvolvidos por Herman Hollerith no contexto das máquinas de tabulação utilizadas para processamento de dados censitários. Posteriormente, por volta de 1923, os algoritmos de ordenação por radix tornaram-se amplamente utilizados para a classificação de cartões perfurados, tecnologia predominante na época para armazenamento e processamento de informações. Hollerith foi também o fundador da empresa *Computing-Tabulating-Recording Company* (CTR), que mais tarde viria a se tornar a IBM (*International Business Machines*), uma das mais influentes empresas da história da computação.

O funcionamento do *Radix Sort* baseia-se na decomposição dos elementos em **posições individuais de dígitos**, processando-os de forma iterativa de acordo com um sistema posicional previamente definido. A ordenação é realizada por meio de sucessivas passagens, nas quais os elementos são agrupados e reorganizados conforme o valor do dígito analisado em cada etapa. Para que o algoritmo funcione corretamente, é essencial que o método de ordenação utilizado em cada passagem seja **estável**, garantindo a preservação da ordem relativa dos elementos com dígitos iguais.

O *Radix Sort* pode ser classificado em dois tipos principais de acordo com a forma como os dígitos são processados:

- ***Least Significant Digit* (LSD – Dígito menos significativo):** Nesse método, a ordenação inicia-se pelo dígito menos significativo (geralmente a unidade) e avança progressivamente em direção ao dígito mais significativo. A cada etapa, os elementos são reordenados com base no dígito corrente, preservando as ordenações anteriores.

- ***Most Significant Digit* (MSD – Dígito mais significativo):** Nesse método, o processo ocorre de forma inversa, iniciando-se pelo dígito mais significativo. A ordenação divide o conjunto de dados em subconjuntos de acordo com esse dígito, aplicando o algoritmo recursivamente aos grupos formados.

Graças à sua eficiência e previsibilidade, o *Radix Sort* é especialmente indicado para cenários que envolvem **grandes volumes de dados numéricos**, como sistemas de processamento de transações, aplicações financeiras, ordenação de identificadores e bases de dados com chaves de tamanho fixo. Contudo, sua aplicabilidade depende diretamente das características dos dados e do custo adicional de memória necessário para as estruturas auxiliares utilizadas durante a ordenação.

Complexidade do *Radix Sort*

O ***Radix Sort*** diferencia-se dos algoritmos de ordenação tradicionais por não utilizar comparações diretas entre elementos. Em vez disso, sua complexidade está associada à quantidade de dígitos (ou caracteres) dos elementos a serem ordenados e à base numérica adotada no sistema de classificação.

A análise de desempenho do *Radix Sort* considera principalmente dois fatores:

- **n**: número de elementos do arranjo;

- **k**: número de dígitos do maior elemento (ou comprimento máximo da chave);

- **b**: base do sistema de numeração (por exemplo, base 10 para números decimais).

Diferentemente de algoritmos baseados em comparação, o *Radix Sort* apresenta o **mesmo comportamento assintótico** nos três cenários, pois o número de passagens depende apenas da quantidade de dígitos e não da disposição inicial dos dados:
A complexidade do *Radix Sort* no **pior caso é** (sendo k o número de dígitos no maior número presente na lista): $O\ (n\ .k)$. No **melhor caso** é: $O\ (n\ .k)$. E no **caso médio**: $O\ (n\ .k)$.

Na primeira etapa, estabelecemos um sistema de classificação com base na quantidade de símbolos compreendidos no sistema posicional. No caso do sistema numérico indo-arábico, levaremos em consideração 10 algarismos (símbolos): [0 ... 10].

	classificação									
símbolo	0	1	2	3	4	5	6	7	8	9
quantidade										
p. inicial										

	a	b	c	d	e	f	g	h
Inicial	104	913	511	4	201	821	17	103

Na sequência, determina-se que a primeira posição de cada valor, partindo o algarismo menos significativo, sendo, nesta etapa, o algarismo referente à unidade. Após isso, realizamos a contagem de cada elemento e somamos o placar da classificação, de acordo com o algarismo identificado.

	classificação									
símbolo	0	1	2	3	4	5	6	7	8	9
quantidade		3		2	2			1		
p. inicial										

	a	b	c	d	e	f	g	h
Inicial	104	913	511	4	201	821	17	103

A terceira e quarta etapa consistem em determinar as posições dos elementos no placar, partindo do menor algarismo até o maior. As posições somente devem ser atribuídas aos algarismos que marcaram pontos na rodada.

A posição inicial é sempre o menor índice do vetor. A posição dos próximos elementos é determinada pela posição do algarismo atual + placar do algarismo atual.

	classificação									
símbolo	0	1	2	3	4	5	6	7	8	9
quantidade		3		2	2			1		
p. inicial		1		4	6			8		

	a	b	c	d	e	f	g	h
Inicial	104	913	511	4	1	821	17	103

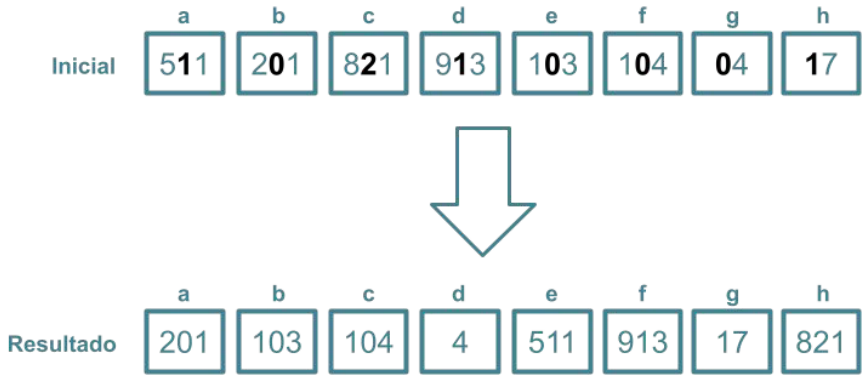
	a	b	c	d	e	f	g	h
Resultado	511	201	821	913	103	104	4	17

Por fim, os elementos são ordenados de acordo com as posições encontradas, encerrando a rodada de ordenação.

As próximas rodadas repetem as mesmas etapas passadas, reiniciando o placar de cada algarismo e suas posições iniciais. As rodadas são executadas até que se atinja o último algarismo significativo (diferente de zero e nulo) do maior valor do arranjo de elementos.

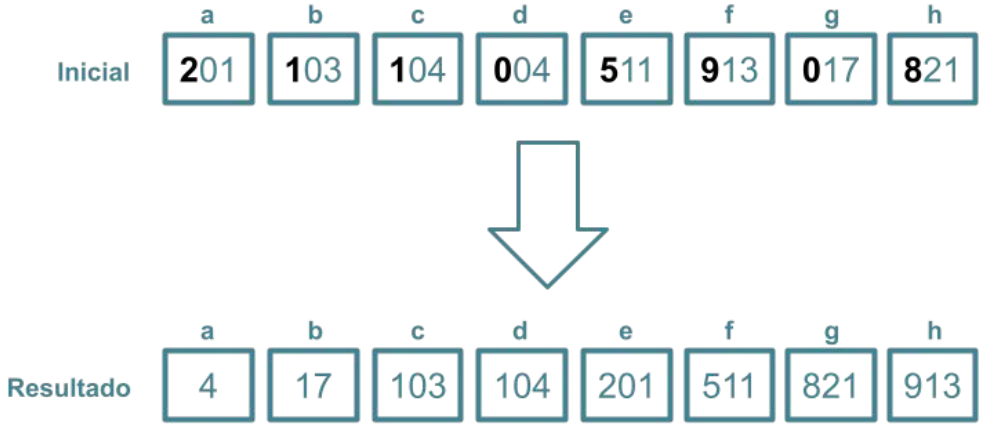
Caso o valor não possua um algarismo significativo na posição indicada na rodada, adota-se o menor algarismo do conjunto de símbolos (geralmente zero para números e nulo para alfabeto).

	classificação									
símbolo	0	1	2	3	4	5	6	7	8	9
quantidade	4	3	1							
p. inicial	1	5	8							



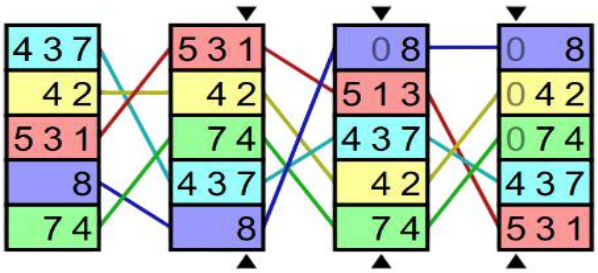
Com a última posição validada, realiza-se a última ordenação dos elementos, obtendo o resultado final da ordenação.

	classificação									
símbolo	0	1	2	3	4	5	6	7	8	9
quantidade	2	2	1			1			1	1
p. inicial	1	3	5			6			7	8



O *Radix Sort* é comumente aplicado nos seguintes cenários:

- Dados Inteiros e *Strings*: Ordenação de números inteiros grandes e strings de comprimento fixo.
- Dados Científicos: Processamento de grandes conjuntos de dados científicos.
- Aplicações Financeiras: Ordenação de grandes volumes de transações financeiras.



CAPÍTULO 4

TABELA DE DISPERSÃO E PESQUISA SEQUENCIAL

Capítulo 4

Tabela de Dispersão e Pesquisa Sequencial

Introdução

Após o estudo dos principais algoritmos de ordenação e das estratégias utilizadas para organizar grandes volumes de dados, este capítulo amplia o foco para outro problema fundamental da Ciência da Computação: a **busca eficiente de informações**. Em aplicações reais, ordenar dados é apenas uma etapa intermediária; o objetivo final, na maioria dos casos, é localizar, recuperar ou acessar informações de forma rápida e precisa.

Neste capítulo, são apresentadas as **tabelas de dispersão (*hash tables*)** e a **pesquisa sequencial**, duas abordagens distintas para acesso a dados, cada uma com características, vantagens e limitações próprias. Enquanto a pesquisa sequencial representa o método mais simples e direto de busca, as tabelas de dispersão introduzem um mecanismo de indexação que permite reduzir significativamente o tempo de acesso aos elementos, mesmo em grandes conjuntos de dados.

Serão discutidos os conceitos fundamentais das tabelas de dispersão, incluindo **funções de espalhamento (*hash*)**, tipos de indexação e estratégias para tratamento de colisões, aspectos essenciais para o correto funcionamento e desempenho dessas estruturas. Além disso, o capítulo aborda a análise de desempenho da pesquisa sequencial, destacando seu comportamento assintótico e os cenários em que sua utilização pode ser adequada.

Ao final deste capítulo, espera-se que o estudante compreenda como diferentes estratégias de busca impactam diretamente o desempenho dos sistemas computacionais, sendo capaz de escolher a técnica mais apropriada de acordo com o contexto da aplicação, o volume de dados e os recursos disponíveis. Esse conhecimento serve como base para o estudo de métodos de

busca mais eficientes, apresentados no capítulo seguinte, e para o desenvolvimento de sistemas que exigem acesso rápido e confiável à informação.

4.1. Tabela de Dispersão

Tabelas de dispersão, também conhecidas como Tabelas *Hashes*, *ou Hash Tables*, são estruturas de indexação de elementos cuja principal finalidade é otimizar a busca de dados em grandes volumes e arranjos.

A história da invenção da tabela de dispersão é incerta. Há duas vertentes, sendo a primeira creditada Hans Peter Luhn em 1953. Luhn foi um pesquisador alemão nos campos da Ciência da Computação, Biblioteconomia e Ciência da Informação para a IBM, sendo mais conhecido por ter sido o criador do algoritmo de Luhn, da indexação KWIC e da divulgação seletiva de informações.

A outra hipótese é de que a Tabela de Dispersão fora inventada pelos autores de compiladores das linguagens de programação da década de 1960, como um truque para manter as listas de variáveis de usuário e seus respectivos valores.

Tabelas *Hashes* são tipicamente usadas para indexação de grandes volumes de informação. Muitos bancos de dados relacionais disponibilizam índices para serem utilizados em colunas a fim de otimizar a busca de informações. Esses índices são conhecidos como "*hash index*".

Alguns interpretadores de linguagens de programação, como o BASIC, utilizam tabelas de dispersão para manter valores de variáveis durante a execução de um programa. Aliás, um interpretador BASIC nada mais é do que uma calculadora com uma tabela de dispersão para controle das variáveis.

Outros exemplos de uso das tabelas de dispersão são as tabelas de transposição em jogos de xadrez para computador até mesmo em serviços de DHCP.

4.2. Função de espalhamento

A **função de espalhamento**, popularmente conhecida como **função *hash*** ou **função modular**, é um mecanismo matemático utilizado para **mapear uma chave** em um **índice** de uma tabela de dispersão. Seu principal objetivo é transformar valores pertencentes a um domínio amplo — como números, strings ou identificadores — em posições específicas de uma estrutura de dados de tamanho limitado, permitindo acesso rápido e eficiente às informações armazenadas.

O termo *função modular* decorre do uso frequente da **operação de módulo (MOD)** no cálculo do índice. Nesse método, escolhe-se um valor base — normalmente relacionado ao tamanho da tabela *hash* — e aplica-se uma divisão inteira à chave. O **resto da divisão** é então utilizado como índice da tabela, conforme a expressão:

$$índice = chave \bmod m$$

em que *m* representa o tamanho da tabela ou um valor base previamente definido.

Uma função de espalhamento eficiente deve distribuir as chaves de forma **uniforme** ao longo da tabela, minimizando a ocorrência de **colisões**, situação em que duas ou mais chaves distintas são mapeadas para o mesmo índice. Quanto melhor for a distribuição dos índices gerados, menor será a probabilidade de colisões e, consequentemente, melhor será o desempenho das operações de inserção, busca e remoção.

É importante ressaltar que nenhuma função *hash* é capaz de evitar completamente colisões, especialmente quando o número de chaves é maior que o número de posições disponíveis na tabela. Por esse motivo, a escolha adequada da função de espalhamento e do valor base *m* — frequentemente um **número primo** — exerce influência direta na eficiência da tabela de dispersão.

Além das funções modulares simples, existem outras técnicas de espalhamento, como:

- funções baseadas em **multiplicação**;
- funções que utilizam **extração ou combinação de dígitos**;
- funções específicas para **cadeias de caracteres (*strings*)**, que consideram o valor numérico dos caracteres e sua posição na chave.

Independentemente da técnica adotada, a função de espalhamento desempenha um papel central no funcionamento das tabelas hash, sendo responsável por garantir acesso rápido aos dados e reduzir o custo computacional das operações de busca, quando comparadas a métodos sequenciais.

Exemplo 1 – Função *hash* modular simples

Considere uma tabela hash com **m = 10 posições**, indexadas de 0 a 9, e a chave:

- **Chave:** 37

Passo a passo:

1. Define-se o tamanho da tabela: **m=10m**
2. Aplica-se a operação de módulo: **37 mod 10 =7**
3. O índice gerado é: **Índice=7**

📌 **Resultado:** o elemento associado à chave 37 será armazenado na posição **7** da tabela *hash*.

Observações Importantes

- A escolha do valor **m** influencia diretamente a eficiência da função hash.
- Bases **primas** são preferíveis para evitar padrões indesejados.
- Mesmo com uma boa função de espalhamento, **colisões são inevitáveis**.
- O objetivo não é eliminá-las, mas **minimizar sua ocorrência**.

4.3. Tipos de Indexação

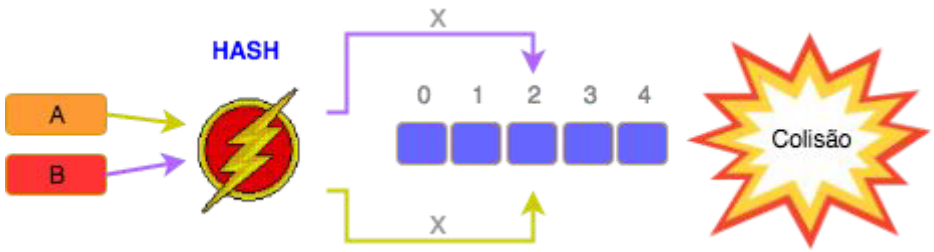
Os dois tipos de indexação mais populares para tabelas *hash* são:

- **Endereçamento Direto:** Cada chave é mapeada diretamente para uma posição única na tabela usando uma função *hash*. É uma abordagem simples, porém ineficiente em casos com chaves muito espaçadas.
- ***Hashing* com Tratamento de Colisões:** Nem sempre é possível evitar colisões (duas ou mais chaves mapeadas em um mesmo índice), porém existem várias técnicas para para lidar com as colisões.
    - ➢ **Encadeamento**: Cada posição na tabela - ou *bucket* - contém uma lista vinculada, onde todas as chaves mapeadas estão armazenadas nessa lista.
    - ➢ **Endereçamento aberto:** A tabela é percorrida de acordo com uma
    determinada estratégia (geralmente linear), até encontrar uma posição vazia.

4.4. *Hashing* com Tratamento de Colisões

No *Hashing* com tratamento de colisões, devemos considerar que em algumas hipóteses, a função *hash* vai encontrar o mesmo índice para mais de um valor.

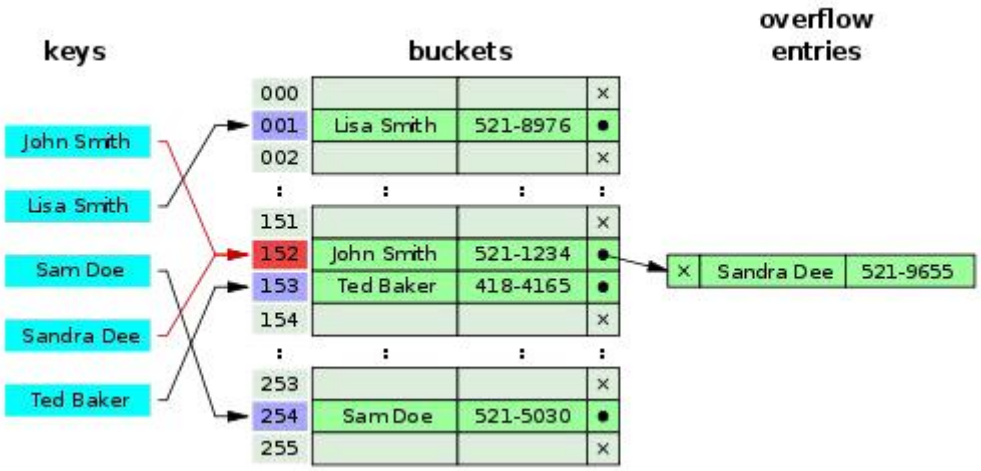
Isso é completamente compreensível, pois mesmo a melhor função *hash*, com a melhor estratégia de cálculo, sempre irá encontrar um índice para mais de um valor. Se levarmos em consideração uma função de *hash* modular, mesmo utilizando um número primo como divisor, como é o caso do número 7, em um universo 10 chaves, de 0 a 10, ainda teríamos 3 colisões.



4.5. *Hashing* com Encadeamento

A primeira forma de se resolver as colisões é implementando o conceito de *buckets*, ou baldes de dados. Nessa técnica, assumimos que cada índice é um balde para armazenar as chaves encontradas, sendo representadas por uma lista encadeada.

Sempre que encontrarmos um índice de um valor que já possui um elemento, adicionamos o outro ao final da lista.

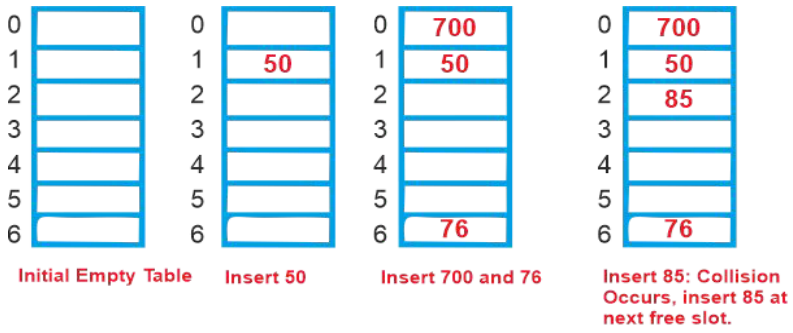


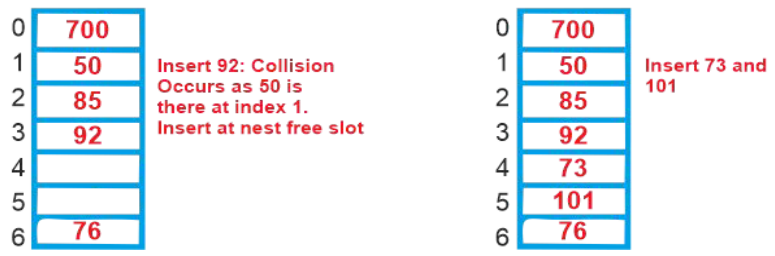
4.6. Hashing com Endereçamento Aberto

Outra forma de resolver as colisões é forçando o endereçamento para uma próxima posição válida. Assim, ao encontrar uma colisão de dados, o algoritmo deve encontrar a próxima posição válida e adicionar o elemento nessa próxima posição.

Caso o algoritmo chegue ao final da tabela, o ponteiro de verificação deve ser direcionado ao início da tabela, procurando a próxima posição válida.

O exemplo abaixo considera a fórmula chave MOD 7 como função *hash*.





4.7. Pesquisa Sequencial

Para que um elemento possa ser localizado em um **arranjo, tabela ou coleção de dados**, é fundamental definir previamente qual **mecanismo de busca** será utilizado. A escolha da técnica de pesquisa impacta diretamente o desempenho do sistema, especialmente quando se trabalha com grandes volumes de informações ou quando o tempo de resposta é um fator crítico.

A forma mais simples e intuitiva de realizar uma busca é por meio da **pesquisa sequencial**, também conhecida como **busca linear**. Nesse método, os elementos da coleção são examinados **um a um**, a partir da primeira posição, até que o elemento procurado seja encontrado ou que se atinja o final da estrutura de dados. Embora esse procedimento pareça óbvio e direto, ele nem sempre é o mais eficiente, principalmente em cenários que envolvem grandes conjuntos de dados.

Apesar de sua simplicidade, a pesquisa sequencial possui grande relevância prática, pois **não impõe qualquer restrição sobre a organização dos dados.** Diferentemente de outras técnicas de busca, ela pode ser aplicada tanto em coleções ordenadas quanto não ordenadas, bem como em estruturas estáticas ou dinâmicas. Essa flexibilidade faz com que a pesquisa sequencial seja amplamente utilizada em aplicações simples, em protótipos ou em situações nas quais o custo de ordenação dos dados não se justifica.

Entretanto, existem diversas outras técnicas de busca que podem apresentar desempenho significativamente superior, dependendo do cenário e da forma como os dados estão organizados. Entre as principais técnicas de pesquisa estudadas neste curso, destacam-se:

- **Pesquisa Sequencial:** percorre os elementos da coleção de forma linear, validando cada item até encontrar o elemento desejado ou esgotar a estrutura.

- **Pesquisa Binária:** utiliza a estratégia de dividir para conquistar, reduzindo o espaço de busca pela metade a cada iteração, sendo aplicável apenas a coleções previamente ordenadas.

- **Pesquisa por Interpolação:** baseia-se em uma estimativa da posição do elemento a partir de uma fórmula matemática, sendo mais eficiente quando os dados estão distribuídos de maneira uniforme.

A compreensão das características, vantagens e limitações de cada uma dessas técnicas é essencial para a escolha do método mais adequado em diferentes contextos computacionais. Nos próximos tópicos, cada uma dessas abordagens será analisada em detalhes, considerando seu funcionamento, desempenho assintótico e aplicações práticas.

Pesquisa Sequencial

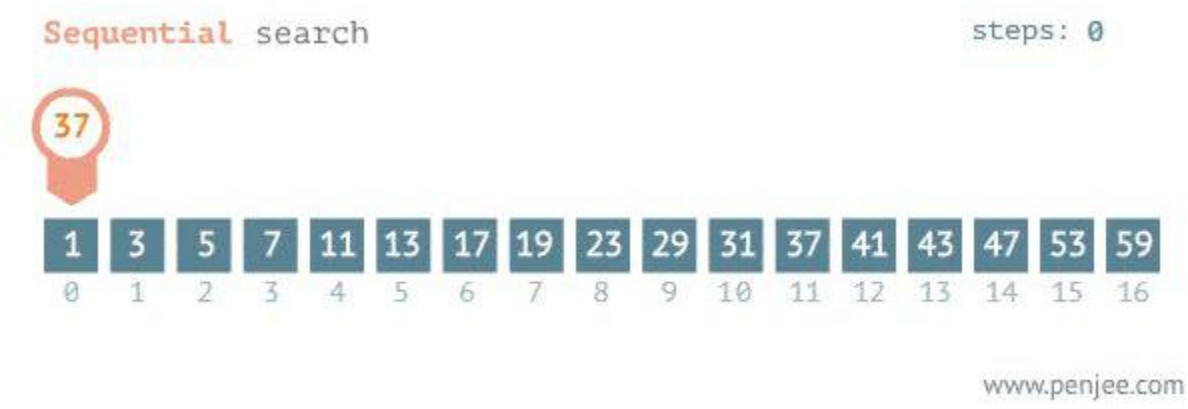
A **pesquisa sequencial**, também conhecida como **busca linear**, é a técnica de pesquisa mais simples e, ao mesmo tempo, uma das mais utilizadas em aplicações computacionais. Nesse método, a estrutura de dados — seja um arranjo, uma tabela ou qualquer tipo de coleção — é percorrida **de forma linear**, elemento por elemento, a partir da primeira posição, até que o valor procurado seja encontrado ou que todos os elementos tenham sido verificados.

Durante a execução da pesquisa sequencial, cada elemento da coleção tem suas informações **validadas individualmente**, sendo comparado com a chave de busca. Caso haja correspondência, o algoritmo encerra sua execução imediatamente, retornando a posição ou o valor encontrado. Se o elemento não estiver presente na coleção, a busca percorre toda a estrutura, atingindo o final do arranjo sem sucesso.

Embora seja extremamente simples de implementar e não exija qualquer tipo de pré-processamento dos dados, a pesquisa sequencial é considerada a técnica **mais onerosa do ponto de vista computacional**. Isso ocorre porque, no pior caso, o algoritmo precisa examinar todos os elementos da coleção, realizando um número de comparações proporcional ao tamanho da entrada. Dessa forma, seu desempenho tende a se degradar significativamente à medida que o volume de dados aumenta.

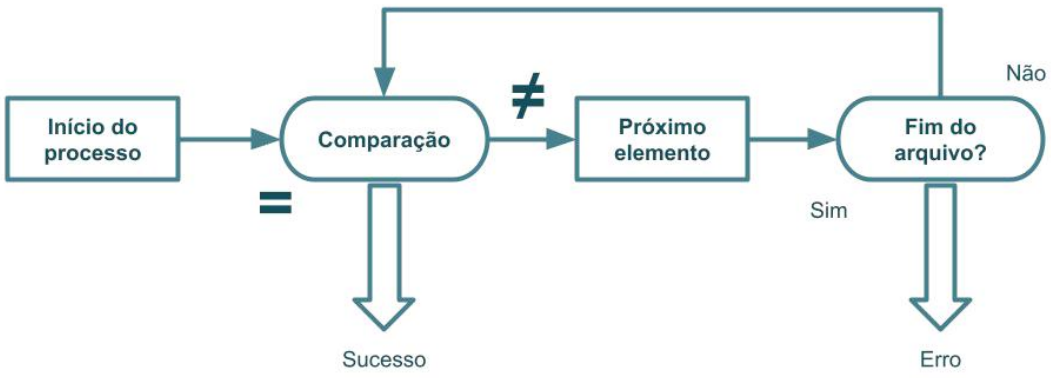
Apesar dessa limitação, a pesquisa sequencial possui algumas vantagens importantes. Ela pode ser aplicada a **dados não ordenados**, não requer estruturas auxiliares e apresenta comportamento previsível. Por esse motivo, é frequentemente utilizada em conjuntos de dados pequenos, em estruturas dinâmicas ou em situações nas quais a simplicidade da implementação é mais relevante do que a eficiência do tempo de execução.

Assim, a pesquisa sequencial desempenha um papel fundamental no estudo das técnicas de busca, servindo como base conceitual para a compreensão de métodos mais eficientes, como a pesquisa binária e a pesquisa por interpolação, que serão abordados nas seções seguintes.



Algoritmo:

Input: A variável que contém o vetor (V) e a chave (K)
Output: -1 se não encontrou o elemento ou o próprio elemento
begin
    forall the elementos de V do
        if elemento = K then
            return elemento
        endif
    endfall
    return -1
end



Análise de Desempenho

A análise de desempenho da **pesquisa sequencial** está diretamente relacionada à quantidade de **comparações** realizadas entre a chave procurada e os elementos da coleção. Como o algoritmo percorre a estrutura de dados de forma linear, seu tempo de execução depende do tamanho do arranjo e da posição do elemento buscado.

O **melhor caso** ocorre quando o elemento procurado encontra-se **na primeira posição** da coleção. Nesse cenário, apenas **uma única comparação** é necessária para localizar o elemento.

$$T(n) = O(1)$$

No **caso médio**, assume-se que o elemento procurado pode estar em **qualquer posição do arranjo com a mesma probabilidade**. Dessa forma, em média, o algoritmo percorre metade da coleção até encontrar o elemento.

$$T(n) = O\left(\frac{n}{2}\right) \Rightarrow O(n)$$

Embora o fator constante 1/2 exista, ele é desprezado na análise assintótica, mantendo-se a complexidade linear.

O **pior caso** ocorre quando o elemento **não está presente** na coleção ou encontra-se **na última posição** do arranjo. Nesse caso, todos os elementos precisam ser verificados, resultando em **n comparações**.

$$T(n) = O(n)$$

Resumindo:

Caso	Número de Comparações	Complexidade
Melhor caso	1	O(1)
Caso médio	n / 2	O(n)
Pior caso	n	O(n)

CAPÍTULO 5

PESQUISA BINÁRIA E PESQUISA POR INTERPOLAÇÃO

Capítulo 5

Pesquisa Binária e Pesquisa por Interpolação

Introdução

Nos capítulos anteriores, foram estudadas diferentes estruturas de dados e técnicas fundamentais para **organização, ordenação e busca de informações**, destacando como essas escolhas influenciam diretamente o desempenho dos sistemas computacionais. A partir desse entendimento, torna-se natural avançar para estruturas que permitem representar **relações mais complexas entre dados**, indo além de coleções lineares.

Neste capítulo, são introduzidas as **estruturas de dados não lineares**, com ênfase em **árvores** e **grafos**, que desempenham papel central em diversas áreas da Computação, como bancos de dados, sistemas de arquivos, redes de computadores, inteligência artificial e análise de caminhos. Diferentemente das estruturas lineares, essas estruturas permitem modelar hierarquias, dependências e conexões entre elementos de forma mais fiel aos problemas do mundo real.

O capítulo aborda os conceitos fundamentais relacionados às árvores, como nós, níveis, altura, grau e tipos de árvores, além das principais operações realizadas nessas estruturas, incluindo inserção, remoção e percursos. Em seguida, são apresentados os grafos, suas representações e aplicações, destacando sua importância na modelagem de redes e sistemas interconectados.

Ao longo do capítulo, será reforçada a análise de desempenho das operações realizadas nessas estruturas, permitindo ao estudante compreender como a escolha adequada da estrutura de dados pode impactar a eficiência e a escalabilidade das aplicações. Ao final, espera-se que o leitor esteja apto a

identificar cenários em que o uso de árvores e grafos é mais adequado, consolidando conhecimentos essenciais para o desenvolvimento de soluções computacionais mais robustas e eficientes.

5.1. Pesquisa Binária

Quando um arranjo ou coleção de dados encontra-se **previamente ordenado**, seja por valores absolutos ou com base em algum **atributo do objeto** que compõe a estrutura, torna-se possível empregar um método de busca consideravelmente mais eficiente do que a pesquisa sequencial: a **pesquisa binária**.

A pesquisa binária baseia-se na estratégia de dividir para conquistar, reduzindo progressivamente o espaço de busca. Em vez de percorrer todos os elementos da coleção, o algoritmo compara inicialmente a chave procurada com o elemento central do arranjo. A partir dessa comparação, a busca é restringida a apenas uma das duas metades do conjunto de dados, descartando completamente a outra metade, que certamente não contém o elemento desejado.

O processo de divisão ocorre de forma recursiva ou iterativa. A cada nova iteração, o arranjo é novamente dividido em dois subarranjos de tamanhos iguais ou muito próximos, e apenas um deles é considerado para a continuidade da busca. Esse procedimento repete-se até que o elemento seja encontrado ou que o subarranjo analisado se torne vazio, indicando que o valor não está presente na coleção.

Uma característica fundamental da pesquisa binária é a necessidade de ordenação prévia dos dados. Caso essa condição não seja atendida, o algoritmo não garante resultados corretos. Em contrapartida, quando aplicada corretamente, a pesquisa binária apresenta uma redução significativa no

número de comparações realizadas, tornando-se extremamente eficiente para grandes volumes de dados.

Dessa forma, a pesquisa binária é amplamente utilizada em sistemas que demandam alto desempenho na recuperação de informações, como bancos de dados, sistemas de indexação e bibliotecas de busca. Nas seções seguintes, serão analisados exemplos práticos, a complexidade assintótica desse método e suas principais vantagens e limitações em relação a outras técnicas de pesquisa.

Exemplo Numérico

Considere o seguinte arranjo ordenado em ordem crescente:

$$A = [3,\ 7,\ 12,\ 18,\ 25,\ 31,\ 42,\ 56,\ 64]$$

Suponha que desejamos localizar o valor **25** utilizando a pesquisa binária.

Passo 1 – Definição dos limites

Inicialmente, são definidos dois índices:

- **início (low)** = 0
- **fim (high)** = 8 (última posição do arranjo)

Calcula-se o índice do elemento central:

$$\text{meio} = \left\lfloor \frac{0+8}{2} \right\rfloor = 4$$

Elemento central:

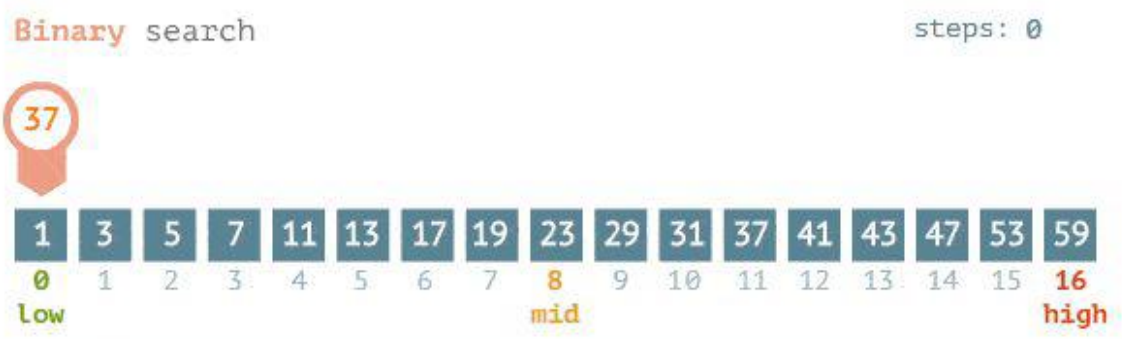
$$A[4] = 25$$

Comparação: 25 = 25 → elemento encontrado.

Neste caso, o elemento foi localizado **na primeira iteração**, caracterizando o **melhor caso** da pesquisa binária.

Observações Importantes

- A cada iteração, o espaço de busca é reduzido aproximadamente pela metade.
- A pesquisa binária é extremamente eficiente para grandes volumes de dados.
- O arranjo **deve estar ordenado** para garantir a correção do algoritmo.

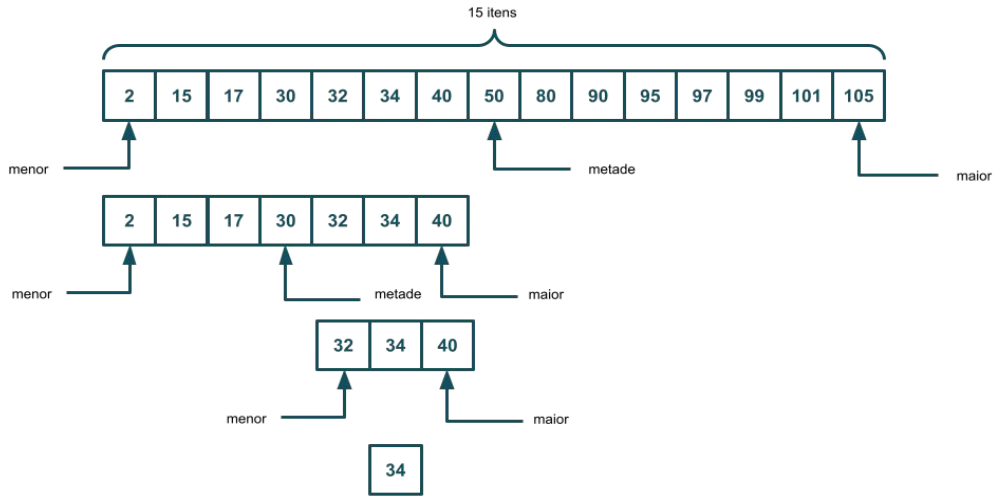


Algoritmo

Input: A variável que contém o vetor (V) e a chave (K)
Output: -1 se não encontrou o elemento ou o próprio elemento
begin

```
baixo ← 1
alto ← quantidade de elementos de V
while baixo ≤ alto do
        meio ← (baixo + alto) / 2
        if K < V[meio] then
                alto ← meio - 1
        else if K > V[meio] then
                baixo ← meio + 1
        else
                return elemento em V[meio]
        endif
endw
return -1
```

end



Análise de Desempenho

A análise de desempenho da **pesquisa binária** baseia-se na quantidade de **comparações** realizadas entre a chave procurada e os elementos do arranjo. Como o algoritmo reduz o espaço de busca pela metade a cada iteração, seu desempenho é significativamente superior ao da pesquisa sequencial, especialmente em grandes volumes de dados.

O melhor caso ocorre quando o elemento procurado encontra-se **exatamente na posição central** do arranjo logo na primeira iteração. Nesse cenário, apenas **uma única comparação** é necessária para localizar o elemento.

$$T(n) = O(1)$$

No caso médio, considera-se que o elemento pode estar em qualquer posição do arranjo ordenado com a mesma probabilidade. A cada iteração, o espaço de busca é reduzido pela metade, o que resulta em um número médio de comparações proporcional ao **logaritmo do tamanho da entrada**.

$$T(n) = O(\log n)$$

O pior caso ocorre quando o elemento não está presente no arranjo ou encontra-se em uma das extremidades. Nessas situações, o algoritmo continuará dividindo o arranjo até que o espaço de busca seja reduzido a um único elemento ou se torne vazio.

$$T(n) = O(\log n)$$

Resumindo:

Caso	Número de Comparações	Complexidade
Melhor caso	1	O(1)
Caso médio	$\log_2 n$	O(log n)
Pior caso	$\log_2 n$	O(log n)

A pesquisa binária apresenta complexidade temporal logarítmica, tornando-se uma das técnicas de busca mais eficientes quando aplicada corretamente. No entanto, sua principal limitação está na necessidade de ordenação prévia dos dados, o que pode implicar um custo adicional quando o arranjo sofre modificações frequentes.

Mesmo assim, em cenários nos quais os dados permanecem ordenados ou sofrem poucas alterações, a pesquisa binária é amplamente utilizada em sistemas que exigem alto desempenho na recuperação de informações, como bancos de dados, sistemas de arquivos e mecanismos de indexação.

5.2. Pesquisa por Interpolação

Dentre as técnicas de pesquisa apresentadas nesta disciplina, a **Pesquisa por Interpolação** destaca-se como uma das mais curiosas e conceitualmente interessantes. Diferentemente da pesquisa sequencial e da pesquisa binária, seu funcionamento não se baseia apenas na divisão sistemática do arranjo, mas na estimativa da posição provável do elemento procurado, utilizando um cálculo matemático conhecido como interpolação — termo que dá nome à técnica.

A ideia central da pesquisa por interpolação é prever, de forma aproximada, em qual posição do arranjo o elemento desejado deve estar considerando a distribuição dos valores presentes na coleção. Em vez de selecionar sempre o elemento central, como ocorre na pesquisa binária, o algoritmo calcula dinamicamente uma posição estimada, buscando reduzir ainda mais o número de comparações necessárias.

O algoritmo da Pesquisa por Interpolação foi proposto em 1957 pelo matemático americano William Wesley Peterson, como uma variação do algoritmo de pesquisa binária. A principal inovação consiste na substituição da escolha fixa do ponto médio por um cálculo que leva em conta os valores armazenados nos extremos do arranjo e o valor da chave procurada.



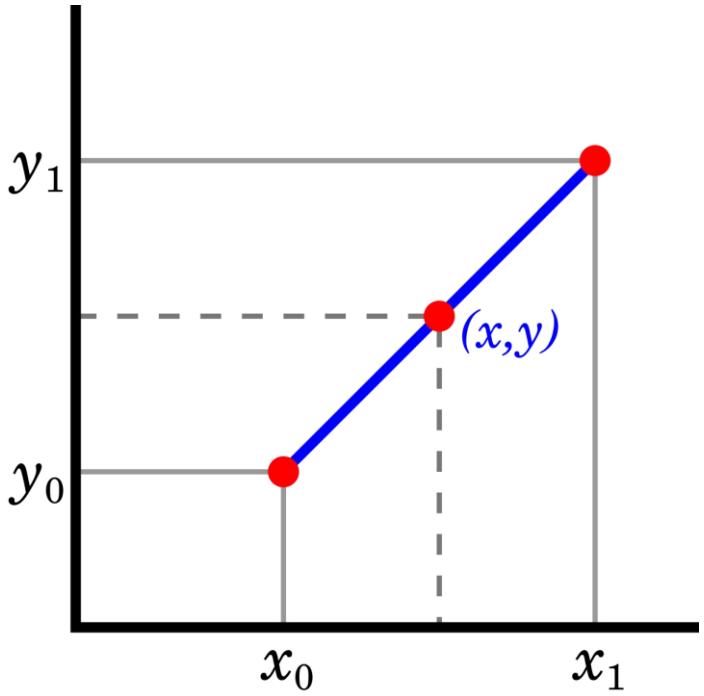
Fórmula

A Pesquisa por interpolação busca prever a posição do elemento buscado de acordo com a fórmula da interpolação linear. Nessa fórmula, estima-se a posição de um elemento (y) dentro de um intervalo definido pelos valores y1 e y2 em relação aos valores correspondentes x1 e x2.

$$y = y_1 + \frac{(x - x_1) \times (y_2 - y_1)}{(x_2 - x_1)}$$

Logo, a fórmula adaptada ao algoritmo de busca seria:

$$posição = baixo + \frac{K - V\,[baixo \times (alto - baixo)}{V\,[alto] - V[baixo]}$$



Assim como a pesquisa binária, a pesquisa por interpolação exige que o arranjo esteja ordenado. No entanto, seu desempenho é fortemente influenciado pela distribuição dos dados. Quando os elementos estão distribuídos de maneira aproximadamente uniforme, a técnica pode localizar o elemento desejado em um número muito reduzido de iterações.

Por outro lado, quando os dados apresentam distribuição irregular ou altamente concentrada, o desempenho da pesquisa por interpolação pode se degradar, aproximando-se do comportamento da pesquisa sequencial. Dessa forma, embora seja extremamente eficiente em cenários específicos, sua aplicação deve ser cuidadosamente avaliada de acordo com o contexto dos dados.

Algoritmo

Input: A variável que contém o vetor (V) e a chave (K)
Output: -1 se não encontrou o elemento ou o próprio elemento

```
begin
    baixo ← 1
    alto ← quantidade de elementos de V
    while baixo ≤ alto do
        meio ← baixo + ((chave - V[baixo]) * (alto -
        baixo) / (V[alto] - V[baixo]))
        if K = V[meio] then
            return elemento em V[meio]
        else if K < V[meio] then
            alto ← meio
        else
            baixo ← meio + 1
        endif
    endw
    return -1
end
```



Análise de Desempenho

A análise de desempenho da **pesquisa por interpolação** está diretamente relacionada à **distribuição dos dados** no arranjo. Diferentemente da pesquisa binária, que sempre divide o espaço de busca ao meio, a pesquisa por interpolação estima a posição do elemento com base em cálculos matemáticos, o que pode reduzir significativamente o número de iterações quando os dados estão distribuídos de forma uniforme.

O melhor caso ocorre quando o cálculo de interpolação estima corretamente a posição do elemento logo na primeira iteração. Nesse cenário, apenas uma comparação é necessária para localizar o valor desejado.

$$T(n) = O(1)$$

No caso médio, assumindo que os elementos estejam uniformemente distribuídos, a pesquisa por interpolação apresenta desempenho superior ao da pesquisa binária. Nessa situação, o número médio de comparações cresce muito lentamente em relação ao tamanho do arranjo, sendo proporcional ao logaritmo do logaritmo do tamanho da entrada.

$$T(n) = O(\log \log n)$$

Esse comportamento torna a pesquisa por interpolação extremamente eficiente em grandes conjuntos de dados bem distribuídos.

O pior caso ocorre quando os dados apresentam uma **distribuição altamente irregular**, com valores muito concentrados em determinadas regiões do arranjo, ou quando a chave procurada está próxima das extremidades. Nessas situações, o algoritmo pode reduzir o espaço de busca de forma pouco eficiente, aproximando-se do comportamento da pesquisa sequencial.

$$T(n) = O(n)$$

A pesquisa por interpolação é uma técnica altamente eficiente quando aplicada a **arranjos ordenados com distribuição uniforme**, podendo superar a pesquisa binária em termos de desempenho. Contudo, sua eficiência depende fortemente das características dos dados, o que limita sua aplicabilidade em cenários genéricos.

Assim, a escolha entre pesquisa sequencial, binária ou por interpolação deve considerar não apenas o tamanho do conjunto de dados, mas também sua organização, distribuição e frequência de atualização, reforçando a importância da análise de desempenho na seleção de algoritmos de busca.

Considerações Finais

Ao longo dos cinco capítulos apresentados neste material, foram abordados os principais conceitos relacionados à **organização, manipulação e recuperação de dados**, elementos fundamentais para a construção de sistemas computacionais eficientes e escaláveis. A progressão dos conteúdos foi estruturada de forma a conduzir o estudante do entendimento básico das estruturas de dados até a aplicação de técnicas mais avançadas de ordenação e busca.

No **Capítulo I**, foram introduzidos os conceitos fundamentais de estruturas de dados, destacando sua importância no contexto da Ciência da Computação e da Engenharia de *Software*. Esse capítulo estabeleceu a base conceitual necessária para a compreensão dos demais temas, enfatizando a relação entre organização dos dados, desempenho e qualidade das soluções computacionais.

O **Capítulo II** apresentou os algoritmos de ordenação mais simples, como *Insertion Sort*, *Bubble Sort* e *Selection Sort*. Embora esses algoritmos possuam baixa complexidade cognitiva e sejam fáceis de implementar, ficou evidente, por meio da análise assintótica, que seu desempenho é limitado, especialmente em cenários com grandes volumes de dados. Essa análise reforçou a importância de compreender não apenas o funcionamento dos algoritmos, mas também seu impacto computacional.

No **Capítulo III**, foram explorados algoritmos de ordenação mais eficientes, como *Quick Sort*, *Merge Sort* e *Radix Sort*, destacando estratégias como **dividir para conquistar** e ordenação por **distribuição**. Esses algoritmos demonstram como abordagens mais sofisticadas podem reduzir

significativamente o tempo de execução, ainda que apresentem maior complexidade de implementação e análise.

O **Capítulo IV** concentrou-se nas técnicas de busca e nas tabelas de dispersão, apresentando mecanismos eficientes de acesso aos dados. A discussão sobre funções de espalhamento, tratamento de colisões e pesquisa sequencial evidenciou como a escolha adequada da estrutura e do método de busca influencia diretamente o desempenho das aplicações.

Por fim, o **Capítulo V** aprofundou o estudo das técnicas de pesquisa, abordando a pesquisa binária e a pesquisa por interpolação. A análise de desempenho dessas técnicas mostrou que, quando aplicadas em cenários adequados, é possível obter ganhos expressivos de eficiência, reforçando a

importância da análise do contexto e das características dos dados antes da escolha do algoritmo.

De maneira geral, este material buscou não apenas apresentar algoritmos e estruturas de dados, mas também desenvolver no estudante a **capacidade de análise crítica**, permitindo compreender as vantagens, limitações e aplicações de cada abordagem. Ao dominar esses conceitos, o leitor estará mais preparado para tomar decisões técnicas fundamentadas, contribuindo para o desenvolvimento de sistemas mais eficientes, robustos e alinhados às demandas do mundo real.

Referências

CELES, Waldemar; CERQUEIRA, Renato; RANGEL, José Lucas. **Introdução a Estrutura de Dados:** Com técnicas de programação em C. São Paulo: Editora CAMPUS. 2011.

LAUREANO, Marcos. **Estrutura de Dados e Algoritmos C**. Rio de Janeiro: Brasport, 2008.

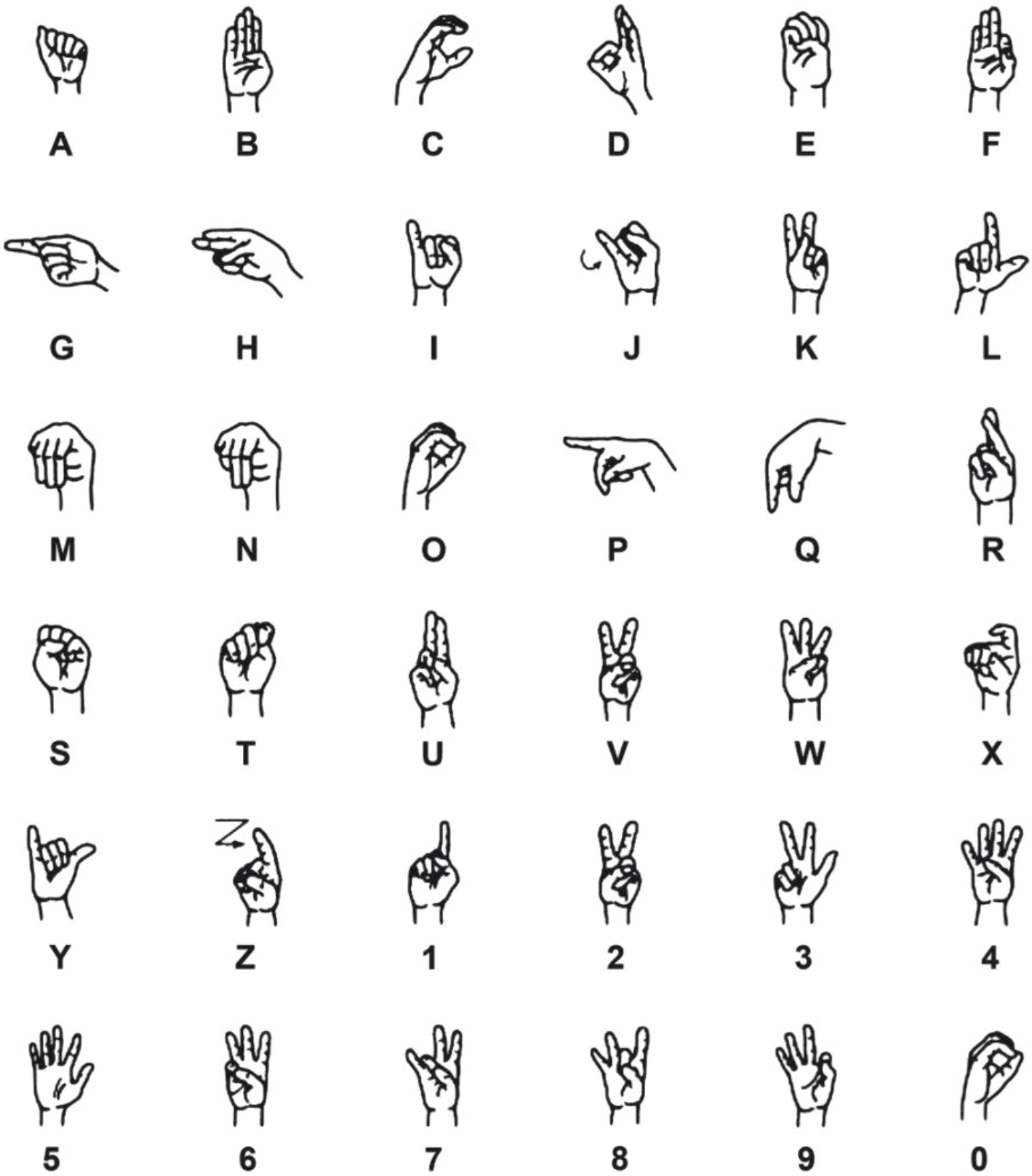
ZIVIANI, Nivio. **Projeto de Algoritmos com implementações em Pascal e C**. 4ª Edição. São Paulo: Pioneira, 1999.

ANICHE, Maurício. **Introdução à programação em C**: Os primeiros passos de um desenvolvedor. São Paulo: Casa do Código. 2015.

GUIMARÃES, A. M, LAGES, N. A. C. **Algoritmos e Estrutura de Dados**. São Paulo: Livros Técnicos e Científicos Editora, 1985.

TANENBAUM, Aron M. **Estrutura de Dados usando C**. São Paulo: MAKRON Books, 1995.

ALFABETO MANUAL



FACULDADE FACINT



Faculdade Facint

Avenida Paraná, número 1118. Primeiro andar. Maringá/PR